Karma Connect & Karma Leads

Technical reference — the product, the architecture, the data, and how to run it without guessing.

36,619	133	21	116	11
LEADS	API ROUTES	TABLES	TESTS	PEOPLE

Karma Staff · internal · 16 chapters
Written from the running system on **1 October 2026** · commit **3579bed**
Regenerate with **npm run docs:pdf**

CONTENTS

PREFACE

About this documentation

The two products in one line each · Reading order · Status of this documentation

PART ONE · THE PRODUCT

1 The product

In one sentence · In one paragraph · In one page · Non-goals · What success looks like · Product principles

2 Features

Karma Leads · Karma Connect · Cross-cutting · What is deliberately absent · How to tell whether the product is working

PART TWO · HOW IT IS BUILT

3 Architecture

The shape of it · Why it is like this · Request routing · Authentication · The Leads server, module by module · The front ends are deliberately different · Scheduled work · Testing

4 The data

How the schema changes · Karma Leads: the tables · What the schema does not enforce · Known data problems · Karma Connect: the tables · Housekeeping that is owed · Queries worth keeping · A caution

5 API

Reading the guard column · Karma Leads — 74 routes · Karma Connect — 62 routes · What this cannot tell you · Routes that are deliberately open

6 Security

Sign-in · Authorisation: a rank ladder, not a set of labels · Row-level access: members see only their own leads · How the 133 routes are actually guarded · Machine callers · HTTP headers · The AI proxy · Secrets · Known gaps

7 External services

Anthropic · Zoho Voice · Apify · The mail gateway · The five public data feeds · Render · GitHub Actions · Adding an integration

PART THREE · RUNNING IT

8 Runbook

The estate · Deploys · Environment variables · Checks · The morning knock · Zoho telephony · Common incidents · Working against production safely · After making a change

PART FOUR · WORKING ON IT

9 Project memory

1. There are two products, two databases, and two user tables · 2. The role ladder is ranked, and duplicated in three places · 3. Data facts that will bite you · 4. Things that look broken but are correct · 5. Zoho is a licence problem, not a code problem · 6. Do not bump the ?v= cache-busters by hand · 7. Render is on the free plan, and it sleeps · 8. Schema changes only via migrations/ + node leads/server/migrate.js · 9. There is a LinkedIn automation bot in agent/ · 10. Known limitations, stated plainly · 11. Do-not-change without understanding

10 Rules for changing this codebase

Before you touch anything · While you work · Rules with teeth · Things that are load-bearing · When you finish · On scope

PART FIVE · DECISIONS

11 **ADR-001**
Merge at the app layer, not the data layer

12 **ADR-002**
The owner tier lives in the environment

13 **ADR-003**
Scheduled work is triggered from outside

14 **ADR-004**
Map geometry is generated offline

PART SIX · WHAT IS STILL OWED

15 **Outstanding**
Written · Next · Corrections made to earlier drafts · Findings that need a decision or an action · Fixed in passing · Deliberately not written, and why

PREFACE

About this documentation

How it was written, and how much to trust it

One Render service serves two products that share a sign-in and a website but not a database. This folder is the source of truth for both.

> **If you are an AI coding agent, read Project memory first.** It is short, and it contains the things that are true about this codebase but not obvious from reading it.

The two products in one line each

	WHAT IT IS	LIVES IN	DATABASE
Karma Connect	AI prospecting: research a prospect, generate outreach, approve and send it	`src/` (React SPA), `server.js`	`karma-connect-db` (Render, free plan)
Karma Leads	The lead base the sales floor works: 36,619 companies, people and job postings, assigned to reps who call them	`leads/` (vanilla SPA, no build step)	`karmaleads_database` (Render, paid)

Both are served by `server.js` on one Render web service. Connect at `/`, Leads at `/leads`. One sign-in cookie covers both.

Reading order

New to the project → The product → Architecture → the half you are working on.

About to change code → Project memory and the `CLAUDE.md` nearest the files you are touching (`leads/CLAUDE.md` is the detailed one).

Operating it → Runbook for what is configured where and what breaks when it is not.

Status of this documentation

> Written from the running system, not from a plan: row counts, route counts and behaviour were read out of the live database and the deployed service. Where something is genuinely undecided it says `DECISION REQUIRED` rather than guessing.

Sections not yet written are listed in Outstanding with what each would need to be worth writing. That file is deliberately honest: half-written documentation is worse than none, because people trust it.

1 The product

What it is, who uses it, what it is not

In one sentence

Karma Staff sells staffing to restoration and construction firms; this system finds those firms, tells a salesperson who to ring and what to say, and records what happened.

In one paragraph

A restoration company — the people who turn up after a fire, a flood or a storm — is busiest exactly when it is least able to hire. Karma Staff supplies them crews. The problem is knowing *which* firm is short-handed *this week*, and getting to them before a competitor. This system holds 36,619 of those firms, assigns them to reps, watches the weather and disaster feeds for where damage just happened, researches a prospect before the call, dials them, records the outcome, and reminds the rep to follow up. Karma Connect is the AI half that writes outreach; Karma Leads is the floor the sales team actually works.

In one page

Who uses it. Eleven people at one company. Three roles:

ROLE	WHO	WHAT THEY DO HERE
member	a sales rep	Works the leads assigned to them: calls, notes, statuses, follow-backs. Sees nothing else.
admin	the sales manager	Runs the floor: assigns work, imports leads, runs job searches, reads the whole team's numbers. Everything an admin can do is recoverable or costs money.
owner	one person	The irreversible things: appointing admins, deleting accounts, emptying the bin, lifting a do-not-call ban.

The core loop.

```
find firms            →  imports, job-board scrapes, storm alerts
  ↓
assign them           →  Assignments board, per rep
  ↓
know who you're ringing → the research briefing: what they do, what to
                          congratulate them on, what not to raise
  ↓
ring them             →  call session: one lead at a time, dial, outcome,
                          note, follow-back — in one press
  ↓
don't lose them       →  follow-back reminder lands in the rep's inbox
  ↓
the manager can see it →  Call report, Team activity
```

What makes it worth building rather than buying. Two things a generic CRM does not do:

1. **Storm watch.** Four official feeds (US National Weather Service, FEMA disaster declarations, NIFC active wildfires, USGS earthquakes) filtered to the ~20% that mean property was damaged, then *joined to the lead base*. It does not answer "what is the weather"; it answers "where did property just get damaged, and who do we already have on file there". A tornado through Tulsa means every restoration firm nearby is turning work away for want of crews, and the first staffing company to ring gets the business.
1. **The briefing.** Before a call, a rep gets what the firm does, anything recent worth congratulating them on, who they are speaking to, and what not to raise — sourced from a LinkedIn scrape, never invented. An empty field is a correct answer; a fabricated one loses the call in ten seconds.

Non-goals

- **Not a CRM.** No pipelines, forecasts, quotes or deals. Statuses are five words and that is deliberate.
- **Not multi-tenant.** One company, one team, one lead base. Nothing is built to be sold to anyone else, and building for that would cost more than it could return.
- **Not a dialler company.** Telephony is Zoho's. This app hands numbers to it and records what came back.
- **Not marketing automation.** Outreach is per-prospect and a human approves every message before it sends.

What success looks like

Not "users" — there are eleven and there always will be. The measures are:

	WHERE YOU CAN ALREADY SEE IT
Calls placed per rep per day	Call report
Connect rate	Call report (needs a Zoho Voice seat — see PROJECT-MEMORY §5)
Leads assigned vs sitting unowned	Assignments; currently ~34k unassigned
Time from a storm to the first call into that state	not measured — **the number this product exists to move**

Product principles

Say what is not known. Every briefing carries a confidence rating, the storm map names feeds that did not answer, the call report says out loud that it only counts calls made through the app. A number without its caveat gets repeated to a customer.

Never make the rep the fallback. When something fails — a licence, a feed, a scrape — the app degrades to something usable and explains it, rather than showing an error and stopping. Copying the number beats a page that says Access Denied.

One press, not four. The call session logs the call, sets the status, writes the note and books the reminder in a single request, because four requests are four ways to end up half-recorded.

Cheap things cached, expensive things asked for. A briefing is a paid web call, so it takes a button and is stored. The storm feed is free, so it refreshes itself every five minutes.

2 Features

What exists, who can reach it, how you would know it works

What exists, who can reach it, and how you would know it is working. This describes the **built** product, not a roadmap — if it is in this file it ships today.

Each feature has an ID so it can be referred to in a commit or a bug report. `KL-` is Karma Leads, `KC-` is Karma Connect.

Acceptance criteria are written as things you can check, not as intentions. Where a criterion is **not** currently met, it says so.

Karma Leads

Thirteen screens. Nine are visible to everyone; four are admin-only, gated server-side, with `.admin-only` in the CSS as presentation only.

ID	SCREEN	TAB	WHO
KL-01	My tasks	`tasks`	everyone
KL-02	Companies	`companies`	everyone
KL-03	People	`people`	everyone
KL-04	Job board	`jobs`	everyone
KL-05	Recent	`recent`	everyone
KL-06	Favorites	`favorites`	everyone
KL-07	Research	`research`	everyone
KL-08	Call session	`camp`	everyone
KL-09	Storm watch	`storm`	everyone
KL-10	Assignments	`assign`	**admin**
KL-11	Call report	`calls`	**admin**
KL-12	Team activity	`stats`	**admin**
KL-13	DNC	`removed`	**admin**

Plus imports, job search and user management, which are actions rather than screens.

KL-01 · My tasks

A rep's queue: the leads assigned to them that are still worth calling.

Acceptance

- A member sees only leads where `leads.owner` equals their email, trimmed.
- An admin sees everything.
- A lead with no phone number does not appear in a calling queue.
- Removed (`removed = true`) and binned (`deleted_at`) leads never appear.

> ⚠️ **Known:** 2,011 leads are owned by `abhimanyu@karmastaff.com`, which has no account, so they appear in nobody's tasks. Not a bug in this feature — see The data.

KL-02 / KL-03 / KL-04 · Companies, People, Job board

The lead base, split by `kind`: 33,043 companies, 2,305 people, 1,271 job postings.

Acceptance

- Paging is **keyset**, so page 400 costs the same as page 1.
- The state filter matches **both** `FL` and `Florida` — counting one spelling under-reports Florida by about 19%.
- Blank strings count as missing everywhere (`nullif(btrim(col), '')`), because importers write `''` as readily as `NULL`.
- A job posting links to its parent company via `company_lead_id`.
- Deleting a company bins its job postings; deleting a job never touches the company.

KL-05 · Recent

The last leads this user opened, per user.

Acceptance

- Scoped to the signed-in user (`recents` is keyed `(user_id, lead_id)`).
- Cleans itself up when a lead is purged — both foreign keys cascade.

KL-06 · Favorites

A flag, indexed partially (`WHERE favorite`) because the set is small.

KL-07 · Research — the prospect briefing

Before a call, what the firm does, anything recent worth congratulating them on, who you are speaking to, and what not to raise. Sourced from a LinkedIn scrape via Apify, drafted through the Anthropic proxy.

Acceptance

- **An empty field is a correct answer.** Nothing is invented — a fabricated detail loses the call in ten seconds.
- Every briefing carries a confidence rating.
- One current briefing per lead, enforced by a partial unique index (`lead_research_current_idx WHERE superseded_at IS NULL`).

- Re-running **supersedes** rather than overwrites, so an old briefing a rep quoted can still be produced.
- It costs money, so it takes a button — never automatic.

Current state: 23 briefings exist. The ceiling is 8,092 — only 22% of leads have a website. Also requires `APIFY_TOKEN`, which is unset, so this is **currently disabled**.

KL-08 · Call session — the dialler

Work a whole assigned list in one sitting: dial, outcome, note, follow-back, next.

Acceptance

- Starting a session **snapshots** the lead ids into `lead_ids bigint[]`. The list cannot shift under the rep as they change statuses — otherwise they skip leads or ring one twice.
- **One open session per person**, enforced by a partial unique index (`call_sessions_open_idx WHERE ended_at IS NULL`), not by the UI.
- `POST /api/campaign/outcome` logs the call, sets the status, writes the note, books the follow-back **and advances** — one request. Four requests are four ways to end up half-recorded.
- Auto-dial: the next number is dialled without a click.
- **Hanging up advances.** Ending a call moves to the next lead; the rep does not press "next".
- The number is copied to the clipboard on every path, including success, so a rep who can see the digits is never stuck.

Current state: works. The *softphone* does not, for licence reasons — see INTEGRATIONS. Copy-to-clipboard is the working configuration today, and that is a deliberate degradation rather than a failure.

KL-09 · Storm watch

A projected map of the US and Canada showing where property was just damaged, joined to the lead base.

Acceptance

- Five official feeds: NWS alerts, FEMA declarations, NIFC wildfires (≥100 acres), USGS earthquakes (≥M4.0), NOAA NHC storms.
- Alerts are **tiered 1–3**, not merely filtered; the map colours on the worst tier per state.
- Marine and lake zones are excluded — a gale warning on Lake Erie is not restoration work.
- Each state shows **how many leads we hold there and how many are unassigned**. That join is the point of the feature.
- **A feed that fails is named.** The response carries a `down` array and the UI shows it. A partial map that says what is missing is honest; a stale one presented as current is not.
- Refreshes every 5 minutes; the feeds are free, so it does not wait to be asked.
- Geometry is real Albers equal-area conic, generated **offline** into a 52 KB `na-geo.js` (ADR-004).

Caveat that must stay visible: roughly a fifth of states in the base were inferred from city or area code at about 96% accuracy. Good enough to decide who to ring; not good enough to quote to a customer.

KL-10 · Assignments *(admin)*

Distribute leads to reps.

Acceptance

- Assignable values come from `GET /api/owners` — accounts **plus** the owner strings already on leads — so one person cannot become three owner filters via a typo.
- An account's assignable value is its **email**, never its display name.
- Bulk assign writes **one** summary row to `activity_log` (`meta.bulk`), not one per lead; 300 rows would bury the feed.

Current state: 33,357 of 36,619 leads are unassigned.

KL-11 · Call report *(admin)*

Calls per rep and connect rate, for any day — plus a 30-day history and a drill-down into one person.

Acceptance

- Reads `activity_log` where `action = 'call'`, served by a partial index `(at DESC, actor) WHERE action = 'call'`.
- **Any past day can be opened**, read from `call_daily`. The numbers no longer reset at midnight.
- **Today is counted live**, every earlier day from the stored rollup, using the same expressions — so the screen and the morning email agree.
- The 30-day strip shows **every** day including empty ones. A gap must read as "nobody called", never as a missing bar.
- A day is a **local** date in `REPORT_TZ`, not a UTC one, or an evening's work lands in the wrong day.
- **Says out loud that it only counts calls made through the app.** A number without its caveat gets repeated to a customer.

Current state: 232 calls over 30 days; history back to 18 Sept.

KL-11b · Downloading the report as a PDF *(admin)*

One Download button in the call report header, opening a small panel: period, detail, go.

Acceptance

- Periods: **this week, last week, this month, last month, custom dates**. Weeks start **Monday** — a sales week does, and a Sunday start files Monday's calls under "last week" every Monday morning.
- Three levels of detail: **totals per person**, **day by day**, **every call** with the company named.
- **Scope follows the screen.** On the team report it exports everyone; with a person's record open it exports just them, and the panel says which ("The whole team" / "Just Surbhi Panwar"). Overall and individual are the same endpoint, so they can never disagree about what a call is.
- Somebody who called nothing is a **zero row, not an absence** — the same rule the morning email follows.
- **Today is rolled up before any export that includes it**, or a range ending today would be short by the day most likely being looked at.

- **PDF**, branded, built with `pdfkit`. Playwright is deliberately not installed on the deploy, so there is no browser on the server to print with; pdfkit is already a runtime dependency and draws to the response stream.
- Four headline numbers, then the table, then a totals row.
- The header names the **period and the scope**, because a file that has been forwarded twice has lost the screen it came from.
- Long company names are **measured and truncated**, not wrapped — rows are a fixed height and a wrapped name prints over the row below.
- CSV remains available at `?format=csv` for anyone who wants to pivot it, with a UTF-8 BOM, formula-injection guards and phone numbers formatted as text.
- **The caveat travels with the file** — the footer rows say it counts only calls placed through the dashboard. A spreadsheet gets mailed on and read by someone who never saw the screen.
- Capped at 20,000 rows; a bigger answer is a query, not a file.

KL-11a · One person *(admin)*

The question a manager asks after "who is calling", which is "what is *this person* doing". Reached by clicking any row in the call report.

Acceptance

- Their daily run for 30 days, the outcome mix, everything else they did (calls are one action of eleven — a rep who placed no calls but wrote thirty notes was not idle), their last 50 calls **with the company named**, their book of leads, and their open follow-ups.
- Days with no calls are included in the average denominator only via `activeDays`, so "per active day" cannot be inflated by a quiet week.
- Works for callers who are **not accounts** — owner strings from the old system made real calls, and hiding them would make totals disagree with rows.
- Admin only: this is a colleague's work in detail, which is exactly what a member may not see.
- A call row links through to the lead, using the same route the activity feed uses, and is plain text when the lead has since been purged.

KL-12 · Team activity *(admin)*

The audit feed: who did what, with before and after values.

Acceptance

- `lead_code` and `lead_name` are denormalised on purpose, so the feed still reads correctly after a lead is purged.
- Machine callers appear as `service:<name>`, distinguishable from people.
- Actions are constrained by a CHECK, so adding one takes a migration.

KL-13 · DNC *(admin)*

The do-not-call list.

Acceptance

- **Removal is not deletion.** `POST /api/leads/:id/remove` blocklists the number and sweeps `removed = true` across **every** lead sharing it.
- Bulk DNC repeats the single-lead rules; it does not shortcut them.
- Lifting a ban is **owner**-only.
- `blocklist` is unique on `phone_key`, the normalised number — not on `phone`.

Current state: 537 blocked numbers, 252 leads flagged `removed`.

KL-14 · Imports

CSV and pipeline ingest.

Acceptance

- Two-phase: rows land in `import_rows` as raw `jsonb`, are previewed, then commit.
- `idempotency_key` is unique, so a retried upload is safe.
- **Blank never overwrites populated.**
- Dedupe is `dedupe.js` and it is identity: franchises are not duplicates. An email match also needs the names to agree (one SERVPRO manager's address sits on nine franchises); a phone match yields when names *and* emails disagree; `pk()` refuses Bitrix placeholder phones, one of which is shared by up to 46 unrelated companies.
- The Python pipeline goes through the HTTP API, never Postgres directly, so it passes the same validation a human upload does.

> ⚠️ **Gap**: committed staging rows are never purged, and two jobs have sat `open` since 11 August. 69 MB of a 114 MB paid database. There is no automatic purge and there should be.

KL-15 · Follow-back reminders

Book a reminder against a lead; it arrives by email when due.

Acceptance

- The email goes to **the person who set it**, not the lead's owner. (It did the wrong thing once: the UI promised "your inbox" while the server used `lead.owner`.)
- At most **one open reminder per lead**, enforced by a partial unique index.
- The sweeper runs every 60 seconds in-process **and** on the external cron knock — the knock is what flushes reminders that fell due overnight while the free-plan process was asleep.

KL-16 · The 7am storm digest

Acceptance

- Sends at `DIGEST_HOUR` local in `DIGEST_TZ`, judged on the real local hour.
- Claimed once a day via `job_runs (job, ran_on)`, so a double knock cannot double-send.
- Claim happens **before** the send, never after: losing a day's digest is a missed email, whereas claiming afterwards risks mailing the whole team twice.

- **A quiet day sends nothing.** A daily "nothing happened" message is one people learn to delete, and then they delete the one that mattered.
- Goes to everyone with a live account, not just managers — a storm is news before it is an assignment.

> ⚠️ **Not currently working.** `job_runs` is empty: no scheduled digest has ever been sent. Diagnosis in RUNBOOK.

KL-16a · The morning call report *(to admins)*

Every morning, yesterday's calling emailed to the people who run the floor: how many calls, how many connected, per member.

Acceptance

- **Rolls yesterday into `call_daily` first, then mails what it just stored.** Not a second query — if the email counted separately it could disagree with the screen an hour later, and then nobody trusts either.
- Goes to **admins and the owner only**. The storm digest goes to everyone because a storm is news; this is every person's numbers side by side, which is a management view. `OWNER_EMAILS` is unioned in, so the owner tier does not depend on a database row.
- **People who called nothing appear as a zero, not as an omission.** A list of only the active makes an idle day look like a quiet one, and the gap is the point.
- Reports **yesterday** — at 7am today has barely started.
- **Sends even on a zero day**, unlike the storm digest. A silent floor is precisely what a manager must be told.
- Compares against the **same weekday last week**, not simply the day before, so "down 60%" does not appear every Monday until people ignore it.
- Claimed once a day via `job_runs`, under its own job name so a storm-digest failure cannot swallow it.
- Says that it counts only calls placed through the dashboard, and names the timezone.

KL-17 · Trash

A 30-day bin.

Acceptance

- Every query over `leads` filters `deleted_at IS NULL`, or binned rows return as phantom duplicates.
- Restore brings back exactly the sweep that was deleted, including a company's job postings (`leads_deleted_company_idx`).
- **Emptying the bin is owner-only and is the one action in the app with no way back.**

Karma Connect

The AI outreach half. Twelve views.

ID	VIEW	WHAT IT DOES
KC-01	Dashboard	the landing summary
KC-02	Board	prospects as a working board
KC-03	Approvals	a member requests a prospect, an admin decides
KC-04	Send queue	outreach waiting to go, with lease-based claiming
KC-05	Coach	a self-tuning playbook learned from what was actually sent
KC-06	History	what happened
KC-07	Reminders	follow-ups
KC-08	Notes	notes against a prospect
KC-09	Team activity	Connect's audit feed
KC-10	Usage	**AI spend per person per day**
KC-11	Admin members	user management
KC-12	Settings	

Acceptance highlights

- KC-04: the queue uses `lease_id` + `lease_expires_at` + `attempts`. A worker that dies mid-send does not strand the row — the lease expires and it is retried; `attempts` stops a poisonous row retrying forever.
- KC-10: `usage_daily` is a rollup keyed `(user_id, day, kind)` with tokens, cost and call count, so AI spend is visible per person per day rather than arriving as a monthly surprise.
- **A human approves every message before it sends.** This is not marketing automation.
- KC-11: appointing an admin and deleting an account are **owner**-only.

Cross-cutting

ID		ACCEPTANCE
X-01	Roles	Three tiers, `owner > admin > member`, checked as "at least". Owner comes from `OWNER_EMAILS`, so a database edit cannot grant it. Verified by a real probe across all three tiers
X-02	Row-level access	A member reaching another rep's lead gets **404, not 403** — a 403 would confirm the lead exists
X-03	Cache-busting	`?v=` is stamped at serve time from the file bytes. Never hand-bumped. It went wrong once and reps ran stale code for days
X-04	KPI tiles	Every tile is a clickable work queue, and the count must equal the rows it opens. The predicate is written in three places and all three must agree
X-05	Degradation	Every integration failure leaves something usable and says what is missing. Copying a number beats a page that says Access Denied

What is deliberately absent

	WHY
Pipelines, forecasts, quotes, deals	Not a CRM. Statuses are five words and that is on purpose
Multi-tenancy	One company, one team, one lead base
A dialler of our own	Telephony is Zoho's. This app hands it numbers and records what came back
Bulk outreach automation	A human approves every message
SMS, push notifications	Email only
A mobile app	Used on desktop by a sales floor

How to tell whether the product is working

Not user counts — there are eleven and there always will be.

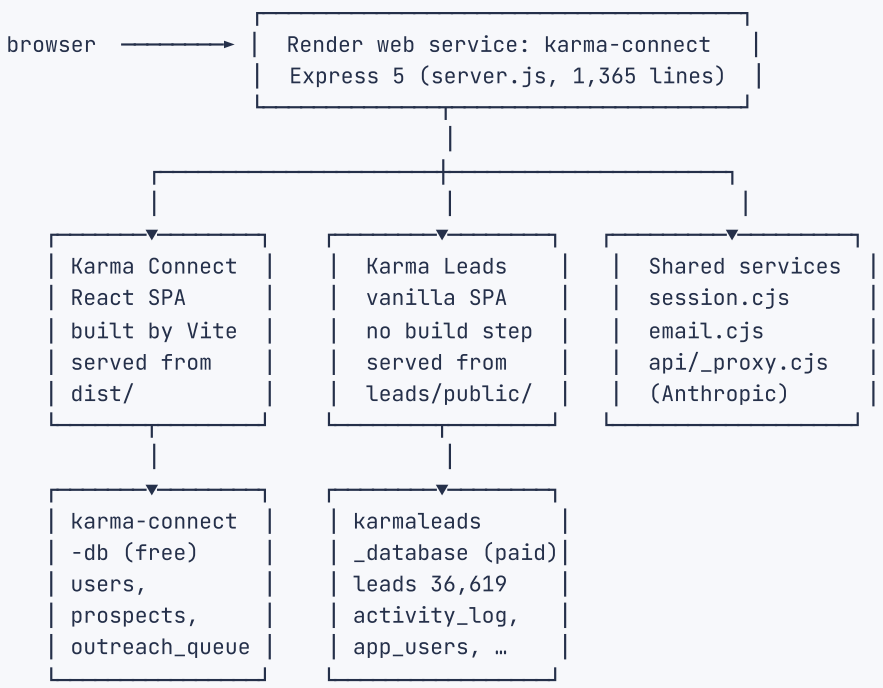
MEASURE	WHERE	TODAY
Calls per rep per day	Call report	124 logged in total
Connect rate	Call report	needs a Zoho Voice seat
Leads assigned vs unowned	Assignments	33,357 unassigned
Statuses moving off `New`	anywhere	13 of 36,619
Time from a storm to the first call into that state	not measured	**the number this product exists to move**

That last row is the honest summary of where the product is: the machinery to move it is built, and the measurement of it is not.

3 Architecture

One process, two products, two databases

The shape of it



One process, two products, two databases. That is the single most important fact about this system and the source of most of its surprises.

Why it is like this

The two products were built separately, each with its own database. They were merged onto one Render service to stop paying for two — and the merge was deliberately **app-layer only**:

> *"i dont want to merge them fulll i want to make a small button which changes it between karma connect and karma lead but i want them both in one website"*

So `leads/server/index.js` exports `mount(app, opts)`, and `server.js` calls it. The Leads API paths are identical whether it runs standalone or merged, because `leads/public/app.js` calls them absolutely. That is what let the front end stay untouched through the cutover.

See ADR-001.

Request routing

PATH	HANDLED BY
`/api/auth/*`, `/api/claude`, `/api/admin/*`, `/api/connect/*`	`server.js` (Connect)
`/api/leads/*`, `/api/counts`, `/api/alerts`, `/api/campaign/*`, …	`leads/server/*` via `mount()`
`/leads/*`	`leads/public/` static, with `index.html` stamped by `cachebust.js`
everything else	Connect's SPA fallback — regex excludes `/api/` and `/leads`

Content-Security-Policy is **scoped per route**: `/leads` gets a policy that allows Zoho's softphone hosts; everywhere else gets a strict one. Likewise `Permissions-Policy: microphone=(self)` is only sent on `/leads`.

Authentication

```
sign in at Connect  →  HS256 JWT  →  httpOnly cookie `kc_session`
                                            |
                                  ┌─────────┴──────────┐
                Connect reads it                        Leads reads it
                (server/auth.cjs)                  (leads/server/auth.js)
                → users table                      → app_users + memberships
```

This replaced WorkOS, which had been set up by someone who left and which nobody remaining could administer — a change as small as allowlisting a callback URL was impossible. WorkOS is still accepted as a fallback so pre-cutover sessions are not thrown out mid-afternoon.

Authorisation is a **rank ladder**: `owner > admin > member`, tested as "at least", never equality. See Security.

The Leads server, module by module

36 modules in `leads/server/`. Grouped by what they are for:

Core data `db.js` · `leads.js` (list, detail, every mutation) · `counts.js` (the KPI tiles, one SQL pass) · `dedupe.js` (identity) · `recents.js` · `trash.js`

Working the base `tasks.js` (a rep's queue) · `assignments.js` · `bulk.js` · `campaign.js` (call sessions) · `calls.js` (call logging + the manager's report) · `followups.js` (reminders + the sweeper)

Bringing data in `imports.js` · `import-worker.js` · `jobsearch.js` (Apify) · `enrich.js` · `webphone.js` (reads phone numbers off company websites)

Intelligence `research.js` + `brief.js` (the prospect briefing) · `linkedin.js` (Apify scrape) · `alerts.js` + `hazards.js` (Storm watch, four public feeds) · `digest.js` (the 7am email)

Plumbing `auth.js` · `service-auth.js` · `users.js` · `activity.js` · `apify.js` · `voice.js` (Zoho) · `cachebust.js` · `migrate.js` · `cli.js` · `demo.js`

The front ends are deliberately different

Karma Connect is React + Vite, built to `dist/`. Conventional.

Karma Leads has no build step. One `app.js` (~5,700 lines), one `app.css`, served as-is. State lives in a single `S` object; `fromApi()` is the whole snake_case → title-case translation in one place; paging is keyset so page 400 costs the same as page 1.

This is not laziness — it is why a change to Leads can be deployed by pushing a file, and why the app has survived being moved between two hosting setups. The cost is no type checking and no module system. Do not "modernise" it without a reason that survives that trade.

Scheduled work

JOB	TRIGGER	WHY NOT A TIMER
7am storm digest	GitHub Actions → `POST /api/cron/daily`	Render free plan sleeps the process; `setInterval` is not running at 7am
Follow-back reminders	in-process every 60s **and** on the cron knock	The knock is what wakes it after a night idle
Trash purge (30 days)	in-process, every 24h	Best-effort; not time-critical
Outreach send tick	in-process (`server/outreach.cjs`)	Only runs while somebody is using the site, which is when it matters

Testing

`vitest`, 116 tests, under a second. The pure, high-consequence logic is extracted so it can be tested without a database or a network: `leads/server/brief.js` (prompt + parsing), `leads/server/linkedin.js` (actor-shape normalising), `src/lib/*.test.js`, `api/_proxy.test.js`, `server/modules.test.js`.

Integration behaviour is verified by booting the API and driving a real browser with Playwright — that is how the role matrix, the call session and the storm map were checked. **Those runs are not in CI**; they are done by hand.

CI does exist — `.github/workflows/ci.yml`, on push to `main` and every PR: `npm ci`, `npm run lint`, `npm test`, `npm run build`. Verified green locally on 2026-09-28 (0 lint errors, 1 warning; 116 tests; clean build).

An earlier draft of this document claimed there was no CI. That was wrong, and it was wrong for the reason AI-RULES warns about — asserted from memory instead of looking in `.github/`.

4 The data

21 tables, the invariants, and the known data problems

Two PostgreSQL databases, one per product, that never join. Row counts here were taken from the live `karmaleads_database` on **2026-09-28** and will have moved; the shapes and the problems will not have.

- `karmaleads_database` — paid plan, **114 MB**, 21 tables, the sales floor
- `karma-connect-db` — free plan, 15 tables, the AI outreach half

They are separate because the products were separate, and the merge was deliberately app-layer only (ADR-001). Nothing in either half reaches across. The cost of that is one real wart: the same person exists in `app_users` *and* in Connect's `users`, added twice, able to disagree.

How the schema changes

This is the one part of the two halves that differs in discipline, not just in style.

	KARMA LEADS	KARMA CONNECT
Mechanism	`leads/migrations/*.sql` + `node leads/server/migrate.js`	`create table if not exists` at boot
Ledger	`schema_migrations` table, 10 rows	none
DDL role	`karma_api`, the only one	the app role
Can you tell what ran?	yes, with timestamps	no

Leads' rule is absolute and is in `leads/CLAUDE.md`: **schema changes only via a migration.** Hand-edits drift and there is no way to notice.

Connect has no migrations. It defines its tables idempotently at start-up, across `server/db.cjs`, `server/outreach.cjs` and `server/linkedin-agent.cjs`, and there is *also* a `db/schema.sql`. Nine tables are defined in both places. See the drift below — it has already happened once.

The migrations, and what each one was for

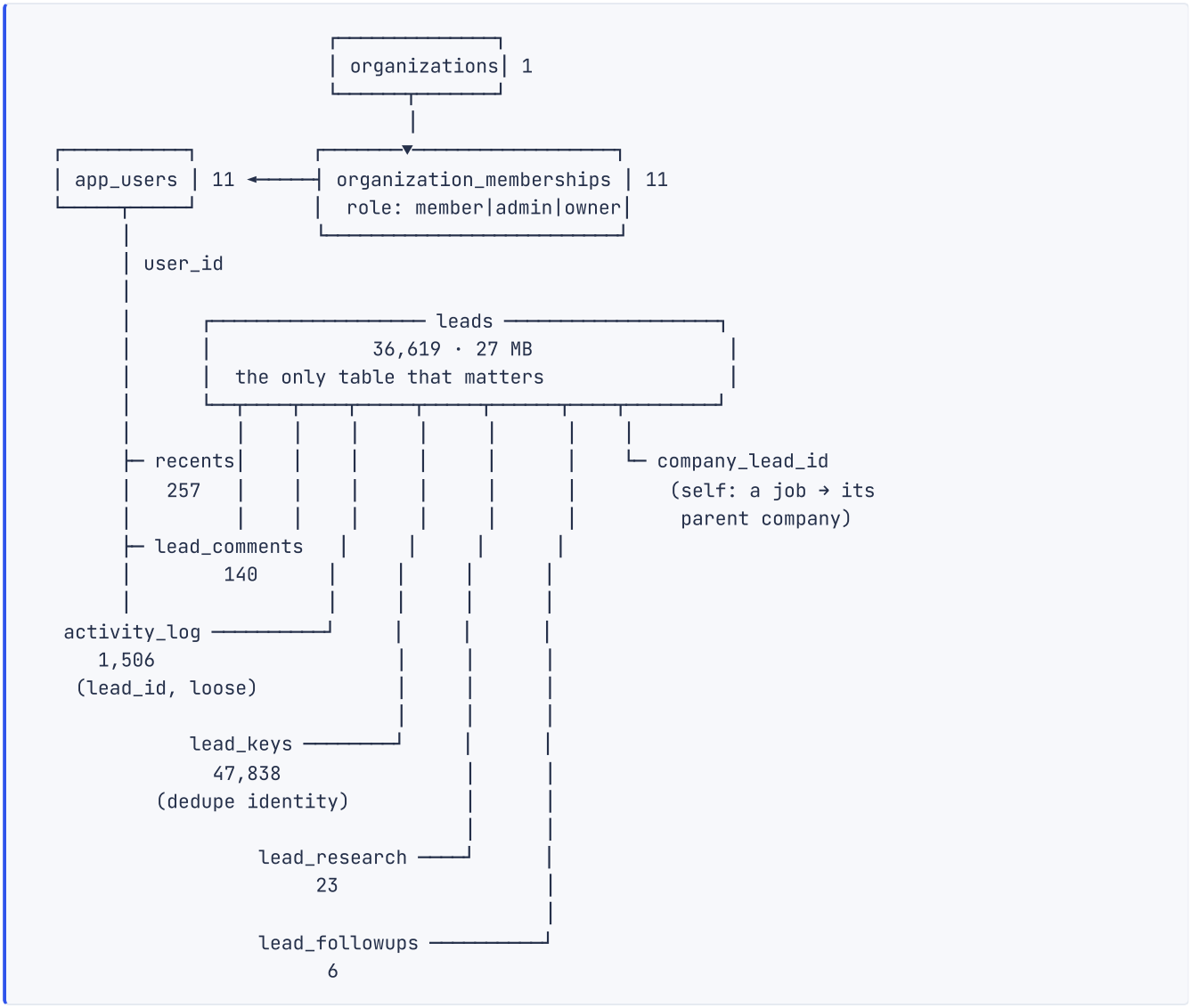
	APPLIED	WHAT IT ADDED
`001_core.sql`	2026-08-11	`leads`, `lead_keys`, `activity_log`, `app_users`, imports, blocklist — the whole original system
`002_company_logo.sql`	2026-08-12	`leads.logo_url`
`003_trash.sql`	2026-08-12	`deleted_at` / `deleted_by`, the 30-day bin

	APPLIED	WHAT IT ADDED
`004_enrich_action.sql`	2026-08-18	`enrich` added to the `activity_log` action CHECK
`005_call_activity.sql`	2026-09-18	call logging: the `call` action + `activity_log_call_at_idx`
`006_owner_role.sql`	2026-09-18	`owner` accepted in `organization_memberships.role`
`007_lead_research.sql`	2026-09-21	`lead_research`, the prospect briefing
`008_followups.sql`	2026-09-21	`lead_followups` + the two partial indexes that make reminders safe
`009_job_runs.sql`	2026-09-21	`job_runs`, so a daily job can be claimed once
`010_call_sessions.sql`	2026-09-22	`call_sessions`, the campaign dialler
`011_call_daily.sql`	2026-09-30	`call_daily` — one row per person per day, so the call report stops resetting at midnight

Migrations 005–010 are all from the last fortnight. That is the shape of this project right now: a stable core from August, and everything since built on top of it.

Karma Leads: the tables

Eighteen live, three dead. Grouped by what they are for rather than alphabetically, because alphabetical tells you nothing.



The core

`leads` — 36,619 rows, 27 MB, 33 columns. Everything else hangs off it.

One table holds three different things, distinguished by `kind`:

KIND	ROWS	WHAT IT IS
`company`	33,043	a restoration or construction firm — the actual target
`person`	2,305	a named contact, sometimes tied to a company
`job`	1,271	a job posting, linked to its company by `company_lead_id`

Notable columns, because the names do not all explain themselves:

COLUMN	WHAT IT REALLY IS
`lead_code`	the human-facing id, unique, what a rep reads out
`phone_key`	the normalised phone; **this is what dedupe and the DNC ban match on**, not `phone`
`owner`	free text, holding an email address. Not a foreign key. See the owner problem
`status`	five words, free text with a CHECK. 36,606 rows are still `New`

COLUMN	WHAT IT REALLY IS
`removed`	a **do-not-call ban**, not a deletion. See below
`deleted_at`	the 30-day bin. Every query must filter this
`company_lead_id`	a job posting's parent company; self-referencing
`certs`	count of certifications — a rough quality signal, used for sorting

Ten indexes, seven of them partial. The partial ones are not premature optimisation; each exists because a specific screen was slow:

```sql
leads_status_idx      ON (status)         WHERE NOT removed        -- the KPI tiles
leads_owner_idx       ON (owner)          WHERE owner IS NOT NULL  -- a rep's queue
leads_phone_key_idx   ON (phone_key)      WHERE phone_key IS NOT NULL -- dedupe + DNC sweep
leads_segment_idx     ON (category,state) WHERE kind = 'company'   -- the segment filter
leads_favorite_idx    ON (favorite)       WHERE favorite           -- 'favourites' is tiny
leads_deleted_idx     ON (deleted_at DESC) WHERE deleted_at IS NOT NULL -- the bin
```

The pattern is worth copying: the interesting rows are always the minority, so the index only covers them.

`lead_keys` — 47,838 rows. The dedupe index. Primary key is `(kind, key_type, key_value)`, so a key can only ever point at one lead. Four key types, and the split tells you what the data actually has:

KEY_TYPE	ROWS	REACH
`phone`	33,189	the strongest signal, and the most common
`email`	11,645	
`namecity`	2,393	the fallback when there is no phone or email
`name`	611	weakest; name alone almost never decides a merge

This table is written by `leads/server/dedupe.js`, which is **identity** and is load-bearing. Read the comments in it before changing anything: franchises are not duplicates, and every clause in there was added because it caught a real bad merge. One SERVPRO manager's email address sits on nine franchises; up to 46 unrelated companies share a single Bitrix placeholder phone number.

`activity_log` — 1,506 rows. Append-only. What happened, who did it, and the before/after values.

ACTION	COUNT
`open`	1,128
`note`	148
`call`	124
`status`	42
`enrich`	23
`jobsearch`	10
`owner`	10

ACTION	COUNT
`remove`	9
`favorite` / `unfavorite`	6 / 4
`delete`, `import`	1 each

Two things to know. First, `lead_id` is a plain column with no foreign key, and `lead_code` / `lead_name` are **denormalised on purpose** — the feed must still read correctly after a lead is purged. Second, the actions are constrained by a CHECK, which is why adding one takes a migration (004, 005).

`activity_log_call_at_idx` is `(at DESC, actor) WHERE action = 'call'` — the manager's call report is the only thing that reads it and it only ever wants calls.

Bulk actions write one summary row, not three hundred. `leads/server/bulk.js` puts the count in `meta.bulk`. Three hundred per-lead rows would bury the feed.

Working the base

`call_daily` — one row per person per day of calling, kept forever (migration 011). The rollup that makes the call report a history rather than a snapshot.

Before it, the report asked `activity_log` for `at ⩾ date_trunc('day', now())` and nothing else, so every number went to zero at midnight and there was no way to ask what yesterday looked like. The rows were never lost — `activity_log` is append-only — but nothing read them, which to the person looking at the screen is the same thing.

Two reasons it is a table and not a better query:

- **A finished day is a fact.** Its numbers cannot change, so recomputing them on every page load is work repeated for an answer already known.
- **The morning email must agree with the screen.** Both read the same stored row, so a manager cannot be told 82 calls in an email and shown 85 an hour later because two queries drifted.

Today is deliberately not stored — it is still changing. `calls.js` counts today live from the log and reads this table for every day before it, using the same expressions in both so the two cannot disagree.

> ⚠️ **The day is a local date, not a UTC one.** `at` is `timestamptz` and Render runs UTC, so `at::date` puts an 8pm Eastern call into *tomorrow* — which silently moves a whole evening of work into the next day's report and makes the morning email wrong. `call_day(ts, tz)` is the single definition of a day; `REPORT_TZ` (falling back to `DIGEST_TZ`) is the timezone. Migration 011 backfilled on `America/New_York`.

Rebuilding a day is idempotent (`ON CONFLICT … DO UPDATE`), which is what lets the nightly job, a retry and a manual backfill all be the same operation. `POST /api/calls/rollup` is the manual lever; it can only restore the truth, because every figure is recomputed from the append-only log.

`call_sessions` — 10 rows. The campaign dialler. The important column is `lead_ids bigint[]`: the session **snapshots** the lead ids at the moment it starts, rather than re-running the filter each time you advance.

That is deliberate. If it re-queried, the list would shift underneath the rep as they changed statuses — they would skip leads or ring one twice. A snapshot plus `pos` means "next" is unambiguous.

```sql
call_sessions_open_idx ON (actor) WHERE ended_at IS NULL
```

A unique partial index, so **one open session per person** is enforced by the database rather than by hope.

`lead_followups` — 6 rows. The follow-back reminder. Two partial indexes, both load-bearing:

```sql
-- at most one open reminder per lead, enforced
lead_followups_open_idx ON (lead_id)
  WHERE sent_at IS NULL AND done_at IS NULL AND cancelled_at IS NULL

-- what the 60-second sweeper reads: only what is still owed
lead_followups_due_idx  ON (due_at)
  WHERE sent_at IS NULL AND cancelled_at IS NULL
```

`owner` is who gets the email. This column caused a real bug: the UI promised "lands in your inbox" while the server used `lead.owner` — so the reminder went to whoever owned the lead, not the rep who set it. Fixed to use the caller's identity. The row at id 5 in the current data is from before that fix.

`recents` — 257 rows. `(user_id, lead_id)` primary key, `touched_at`. Both foreign keys cascade, so it cleans itself up.

`blocklist` — 537 rows. The do-not-call list, unique on `phone_key`.

Removal is not deletion, and this trips people up. `POST /api/leads/:id/remove` is a *phone ban*: it blocklists the number and sweeps `removed = true` across every lead sharing it. `DELETE` is the 30-day bin. A lead can be removed and not deleted, which is the normal case — 252 rows are currently `removed`, and `0` are in the bin.

Ingest

`import_jobs` (7) + `import_rows` (124,486, **69 MB**) — a two-phase import. Rows land in `import_rows` as raw `jsonb`, get validated and previewed, and only then commit into `leads`. `import_rows` cascades on job delete. `idempotency_key` is unique, which is what makes a retried upload safe.

`leads/server/imports.js` is the **only** bulk ingest path. The Python pipeline reaches it over HTTP via `KARMA_API_URL` / `KARMA_API_TOKEN` and never touches Postgres directly. Blank values never overwrite populated ones.

> ⚠️ **69 MB of the 114 MB database is abandoned import staging.** See housekeeping.

`service_tokens` (1) — hashed tokens for the Python pipeline, with a `scopes` array and a `revoked_at`. Only the hash is stored.

Intelligence

`lead_research` — 23 rows. The prospect briefing, stored as `jsonb`, with the `model` that produced it and the `actor` who asked.

```
lead_research_current_idx ON (lead_id) WHERE superseded_at IS NULL
```
SQL

Unique and partial: **one current briefing per lead**, with the old ones kept. A re-run stamps `superseded_at` rather than overwriting, so a rep who quotes an old briefing can be shown what it actually said.

Briefings are stored because they cost money to produce — a paid web call per lead. Cheap things refresh themselves; expensive things get a button and a row.

`job_runs` — `(job, ran_on)` primary key, so a daily job is claimed once even if the trigger fires twice. Written by `leads/server/digest.js`.

> ⚠️ **This table is empty.** No daily digest has ever recorded a successful run. See housekeeping.

Accounts

`app_users` (11) · `organizations` (1) · `organization_memberships` (11). One organisation. Role lives on the membership, not the user:

ROLE	COUNT
`member`	7
`admin`	3
`owner`	1

`owner` became a legal value in migration 006. Note that the **owner tier is not defined by this table alone** — `OWNER_EMAILS` in the environment can confer it, which is what stops a database edit from granting the irreversible powers. See ADR-002.

`app_users.oidc_sub` is a leftover of the WorkOS era, nullable and mostly unset.

Dead tables

TABLE	ROWS	WHAT IT IS
`notification`	1	NocoDB residue
`workspace`	1	NocoDB residue — has Stripe columns, grace periods, plan tiers
`workspace_user`	1	NocoDB residue — SCIM columns
`lead_code_aliases`	0	ours, built for a rename path that was never used

The first three are from when this database was behind NocoDB. Nothing in our code reads them — grep for the names and you get nothing. They are harmless but they are confusing: `workspace.stripe_customer_id` in particular has made more than one person ask whether this product takes payments. It does not.

`lead_code_aliases` is ours and cascades correctly; it is simply unused.

What the schema does not enforce

These are the invariants the application has to keep. A constraint cannot express any of them, which is why they are written down.

1. Every query over `leads` filters `deleted_at IS NULL`. Forget it and binned leads come back as phantom duplicates. There are `0` binned rows today, which makes this easy to forget and no less true.

2. Deleting a company bins its job postings. Deleting a job never touches the company. `leads_deleted_company_idx` on `(company_lead_id, deleted_at)` is what makes restoring that exact sweep possible.

3. The KPI tile predicates are written three times — `FOCUS` in `leads/server/leads.js`, the `FILTER` clauses in `leads/server/counts.js`, and `FOCUS_TEST` in `leads/public/app.js`. Change one, change all three. A tile that says 412 and opens 380 rows is worse than no tile.

Blank strings count as missing throughout, because importers write `''` as readily as `NULL`. Hence `nullif(btrim(col), '') IS NULL` rather than `col IS NULL`, everywhere.

4. Bulk actions repeat the single-lead rules; they never shortcut them. The bulk DNC still blocklists each number and still sweeps every lead sharing it.

5. Counting by state must match both spellings. See below.

Known data problems

Not bugs to be silently "fixed" — these are properties of the data, and code that pretends otherwise reports wrong numbers.

State is inconsistent, and a fifth of it is inferred

	ROWS
two-letter code (`FL`)	31,623
spelled out (`Florida`)	3,335
blank	1,661

Florida alone, right now:

STORED VALUE	ROWS
`FL`	2,585
`Florida`	613
`Fl`	1

Counting only `FL` under-reports Florida by 614 — about 19%. Every state-aware query handles this, and they should keep handling it:

- `leads.js` filter: `(upper(state) = $1 OR state ILIKE $2)`

- `alerts.js` `leadCounts()`: matches both, on `upper(btrim(state))`, then folds `FLORIDA` back onto `FL`
- `enrich.js`: compares on the abbreviation

Two further wrinkles. `upper()` covers the `Fl` row. And the base contains **non-US rows** — one `France`, one `Federal Territory of Kuala Lumpur` — which no state filter will ever match and which quietly sit outside every regional count.

Worse, and undocumented anywhere in the code: roughly a fifth of states were inferred, not sourced. The Python `backfill_states()` fills blanks from city, else from area code: 86% coverage at 96% accuracy. **Expect about 1 in 25 inferred states to be wrong.** Storm watch joins alerts to leads by state, so this feeds directly into "who do we ring after the tornado". It is good enough for that and it is not good enough to quote to a customer.

The `owner` column is free text

`leads.owner` holds an email address as text, matched on equality. There is no foreign key. That was a pragmatic choice — leads were assigned before accounts existed — and it has one live consequence:

> `abhimanyu@karmastaff.com` **owns 2,011 leads and has no account.**

Verified: that address appears in no row of `app_users`. So those 2,011 leads — **6% of the base, the second-largest assignment after unassigned** — are assigned to nobody who can sign in. No rep sees them in their queue. The one follow-up reminder booked against that owner (row id 5) can never reach anyone.

Current distribution:

OWNER	LEADS	HAS AN ACCOUNT?
(unassigned)	33,357	—
abhimanyu@	**2,011**	**no**
ethina@	500	yes, member
abhilasha@	300	yes, member
simran@	200	yes, member
surbhi@	200	yes, member
tenzinwoesel@	25	yes, owner
anjan@	25	yes, admin
pema@	1	yes, admin

Four of the eleven accounts own nothing.

The guard against this getting worse is `GET /api/owners`, which returns accounts plus the owner strings already on leads, so one person cannot become three owner filters through a typo. **An account's assignable value is its email, never its display name.**

> `DECISION REQUIRED` — what happens to abhimanyu@'s 2,011 leads. Either create the account or reassign them. Right now they are dark inventory: the largest single block of assigned leads in the system and nobody is working it.

99.96% of leads are still `New`

STATUS	ROWS
`New`	36,606
`Contacted`	9
`Not interested`	4

Two of the five statuses have never been used. This is not a data bug — the calling workflow is days old — but it means **any funnel metric built on `status` today is measuring nothing**, and the call report's own numbers (124 logged calls) are the only real activity signal.

Contactability

	ROWS	SHARE
have a phone number	32,039	87%
have an email	11,737	32%
have a website	8,092	22%
banned (`removed`)	252	0.7%

Phone is the channel this base supports. That is why the product is a dialler and not an email tool, and why `phone_key` carries the dedupe.

Only 22% have a website, which caps `webphone.js` (phone-number scraping) and the LinkedIn research path — a briefing cannot be produced for a firm with no web presence. 23 briefings exist against 8,092 possible.

Karma Connect: the tables

Fifteen tables on the free plan. Not queried live for this document — local `DATABASE_URL` is blank — so the following is **from source** (`db/schema.sql`, `server/db.cjs`, `server/outreach.cjs`, `server/linkedin-agent.cjs`) and has no row counts.

TABLE	PURPOSE
`users`	Connect's own accounts. `password_hash`, `email_verified`, `is_admin`, reset tokens
`prospects`	a saved prospect: `label`, `sub`, and a `data` jsonb blob
`lead_approvals`	a member requests a prospect; an admin decides. `assigned_admin_id`

TABLE	PURPOSE
`lead_notes`	notes against a prospect
`campaigns` → `campaign_steps` → `campaign_enrollments`	multi-step outreach sequences
`message_sends`	what was actually sent, including the text, so the coach can learn wording
`coach_playbook` / `coach_changes`	the self-tuning outreach coach and its audit trail
`outreach_queue`	the send queue. Leased: `lease_id`, `leased_at`, `lease_expires_at`, `attempts`
`usage` / `usage_daily`	Anthropic token and cost accounting, per user per day per kind
`activity`	Connect's activity feed
`linkedin_agent`	LinkedIn automation state

Two patterns worth noting because they are good:

`usage_daily` is a rollup keyed `(user_id, day, kind)` carrying `input_tokens`, `output_tokens`, `cache_read_tokens`, `cost_usd`, `calls`. AI spend is visible per person per day without scanning the raw `usage` table. This is what makes the proxy's cost recording useful rather than decorative.

`outreach_queue` **uses lease-based claiming** — `lease_id` + `lease_expires_at` + `attempts`, which is the `FOR UPDATE SKIP LOCKED` pattern done properly. A worker that dies mid-send does not strand the row; the lease expires and it is picked up again. `attempts` is what stops a poisonous row retrying forever.

`is_admin` on `users` is a **boolean**, where Leads has a three-tier ranked ladder. The two halves genuinely have different authorisation models.

The Connect drift is real

Nine tables are defined in both `db/schema.sql` and `server/db.cjs`. I compared all nine, ignoring comments:

- `lead_notes`, `usage_daily`, `campaigns`, `campaign_steps`, `campaign_enrollments`, `coach_playbook`, `coach_changes` — **identical**
- `message_sends` — **genuinely differs.** `db/schema.sql` declares `message`, `personalization` and `acceptance`; the `db.cjs` `create table` does not, and three `alter table … add column if not exists` statements (`server/db.cjs:132-134`) patch them back on afterwards.

So the drift already happened, and the boot path papers over it. Worse, `db/schema.sql` is **not a complete schema** — `activity`, `linkedin_agent` and `outreach_queue` exist only in code, so a database built from that file alone would be missing the entire send queue.

> `DECISION REQUIRED` — Connect's schema management. Options: (a) adopt the Leads migration runner for Connect too, (b) delete `db/schema.sql` and make the boot path the single source of truth, (c) leave it. Recommended **(b) now, (a) later** — (b) is an afternoon and removes a file that is already lying; (a) is the right end state but nothing is on fire.

Housekeeping that is owed

Three concrete items, all verified, in order of size.

1. 69 MB of abandoned import staging — 60% of the database.

JOB	STATUS	STAGED ROWS
`sync 20260811-1106`	**open**	27,000
`sync 20260811-1108`	**open**	27,000
`sync 20260811-1109`	committed	35,242
`sync 20260811-1124`	committed	35,242
`pipeline-test.csv`	committed	2

Two jobs from 2026-08-11 are still `open` — started, never committed, never aborted — holding 54,000 staging rows. Two more are committed but their staging rows were never purged. Deleting the job rows cascades to `import_rows` and reclaims roughly 69 MB of a 114 MB paid database.

Do not do this blind: confirm the two committed syncs actually landed (35,242 staged against 36,619 live leads is consistent, but confirm) and confirm nobody intends to resume the open ones. Then delete. There is no automatic purge for this and there probably should be.

2. `job_runs` is empty — the 7am digest has never recorded a run.

The table exists (migration 009, 2026-09-21), `digest.js` writes to it, and it has zero rows.

Read `runDigest()` carefully before concluding it is broken. The day is claimed **after** the hour check and the mail check and **before** the send, and a `force`d run never claims at all. So an empty `job_runs` is consistent with any of:

1. the GitHub secrets are unset, so the workflow emits a notice and never knocks
2. `CRON_TOKEN` is unset on Render, so the knock gets a 503
3. the knock arrives but the local hour is not `DIGEST_HOUR` — a correct skip
4. mail is not configured (`GATEWAY_URL` / `GATEWAY_TOKEN`) — also skipped early
5. every run so far was `force`d, which deliberately leaves no row

What it does establish is that **no scheduled digest has ever been sent.** That matters more than the empty table. Diagnosis is in Runbook; the check is one query:

```sql
SELECT * FROM job_runs ORDER BY ran_at DESC LIMIT 5;   -- currently: nothing
```

3. Three dead NocoDB tables. `notification`, `workspace`, `workspace_user`. One row each, read by nothing. Dropping them is a migration and it is safe; leaving them costs nothing but keeps inviting the question about Stripe.

Queries worth keeping

Diagnostics that have each earned their place. Run them read-only.

```sql
-- assigned to somebody who cannot sign in (found the 2,011)
SELECT l.owner, count(*) FROM leads l
 WHERE l.owner IS NOT NULL AND l.deleted_at IS NULL
   AND NOT EXISTS (SELECT 1 FROM app_users u
                   WHERE lower(u.email) = lower(l.owner))
 GROUP BY 1 ORDER BY 2 DESC;

-- is a state split across spellings?
SELECT state, count(*) FROM leads
 WHERE deleted_at IS NULL AND state ILIKE 'f%'
 GROUP BY 1 ORDER BY 2 DESC;

-- did the daily job run?
SELECT job, ran_on, ran_at, detail FROM job_runs
 ORDER BY ran_at DESC LIMIT 10;

-- sessions left open (should be at most one per person, and none overnight)
SELECT actor, started_at, pos, array_length(lead_ids,1) AS total
  FROM call_sessions WHERE ended_at IS NULL;

-- reminders that are due and have not gone out
SELECT id, owner, due_at FROM lead_followups
 WHERE sent_at IS NULL AND cancelled_at IS NULL AND due_at < now()
 ORDER BY due_at;

-- what is actually taking up the paid disk
SELECT relname, pg_size_pretty(pg_total_relation_size(c.oid)) AS total
  FROM pg_class c JOIN pg_namespace n ON n.oid = c.relnamespace
 WHERE n.nspname = 'public' AND c.relkind = 'r'
 ORDER BY pg_total_relation_size(c.oid) DESC LIMIT 10;
```

A caution

This repository's development database **is** production. There is no staging copy of the 36,619 leads. Two consequences:

1. A test row you write is a real row. Clean it up, and prefer a distinctive name so it can be found again.
2. `migrate.js` runs against live data. Read the migration before applying it, and remember that the paid plan's backups are the only undo.

5 API

133 routes, generated from the source

136 routes, generated from the source on 2026-10-01.

Every path is absolute. That is deliberate: Karma Leads' API paths are the same whether it runs standalone or mounted into Connect, which is what let its front end survive the merge untouched (ADR-001).

Verified, not just parsed. On 2026-09-28 all 43 parameterless `GET` paths below were probed against a running server: **no 404s**, so the extraction is not inventing routes. (Several returned 500 locally because that instance had no Connect database and no Apify token — a configuration state, not a bad route.)

Reading the guard column

GUARD	MEANING
(blank)	no authentication — check the handler before assuming that is wrong
`signed in`	any valid `kc_session`
`own lead`	a member may only reach leads assigned to them; **404, not 403**, so they cannot learn the lead exists
`admin`	rank ≥ admin. A **rank comparison**, never `=== "admin"`
`owner`	rank ≥ owner. Conferred by `OWNER_EMAILS` (ADR-002)
`service token`	a machine caller, hashed token in `service_tokens`
`rate-limited`	carries an explicit limiter

A guard marked *(inherited)* comes from a `router.use` on a path prefix rather than from the route's own argument list — the protection is real, it is just declared once for the group.

Guards applied to whole prefixes

PREFIX	GUARD	WHERE
`/api/apify-usage`	admin	`leads/server/jobsearch.js:33`
`/api/bulk`	admin	`leads/server/bulk.js:38`
`/api/comments`	signed in	`leads/server/leads.js:28`
`/api/job-search`	admin	`leads/server/jobsearch.js:33`
`/api/leads`	signed in	`leads/server/leads.js:28`
`/api/leads/:id`	own lead	`leads/server/leads.js:55`
`/api/phone-refresh`	admin	`leads/server/webphone.js:123`
`/api/recents`	signed in	`leads/server/recents.js:16`

PREFIX	GUARD	WHERE
`/api/users`	admin	`leads/server/users.js:22`

Karma Leads — 74 routes

`/api/activity`

METHOD	PATH	GUARD	SOURCE
GET	`/api/activity`	admin	`leads/server/index.js:130`

`/api/alerts`

METHOD	PATH	GUARD	SOURCE
GET	`/api/alerts`	signed in	`leads/server/alerts.js:215`

`/api/apify-usage`

METHOD	PATH	GUARD	SOURCE
GET	`/api/apify-usage`	admin *(inherited)*	`leads/server/jobsearch.js:696`

`/api/assignments`

METHOD	PATH	GUARD	SOURCE
GET	`/api/assignments`	admin	`leads/server/assignments.js:29`
GET	`/api/assignments`	admin	`leads/server/demo.js:241`

`/api/auth`

METHOD	PATH	GUARD	SOURCE
GET	`/api/auth/callback`	—	`leads/server/auth.js:316`
GET	`/api/auth/callback`	—	`leads/server/auth.js:394`
GET	`/api/auth/login`	—	`leads/server/auth.js:306`
GET	`/api/auth/login`	—	`leads/server/auth.js:393`
POST	`/api/auth/logout`	—	`leads/server/auth.js:355`
POST	`/api/auth/logout`	—	`leads/server/auth.js:395`

`/api/bulk`

METHOD	PATH	GUARD	SOURCE	
POST	`/api/bulk/assign`	admin *(inherited)*	`leads/server/bulk.js:83`	The owner column is free text (it holds email addresses today, but the old system wrote plain…
POST	`/api/bulk/assign`	admin	`leads/server/demo.js:325`	
POST	`/api/bulk/delete`	admin *(inherited)*	`leads/server/bulk.js:169`	The cascade rule from trash.js, unchanged:
POST	`/api/bulk/dnc`	admin *(inherited)*	`leads/server/bulk.js:118`	Same contract as the single-lead ban:
POST	`/api/bulk/preview`	admin *(inherited)*	`leads/server/bulk.js:51`	One pass answers both destructive modals:

`/api/calls`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/calls/export`	admin	`leads/server/calls.js:570`	
GET	`/api/calls/person/:email`	admin	`leads/server/calls.js:268`	GET /api/calls/person/:email — one person, in depth.
GET	`/api/calls/report`	admin	`leads/server/calls.js:141`	GET /api/calls/report — the manager's screen.
POST	`/api/calls/rollup`	admin	`leads/server/calls.js:390`	POST /api/calls/rollup — rebuild stored days.

`/api/campaign`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/campaign/current`	signed in	`leads/server/campaign.js:135`	
POST	`/api/campaign/end`	signed in	`leads/server/campaign.js:217`	
POST	`/api/campaign/outcome`	signed in	`leads/server/campaign.js:147`	One press: log the call, set the status, optionally book a follow-back and leave a note, then…
GET	`/api/campaign/preview`	signed in	`leads/server/campaign.js:82`	How many are callable, before committing to a session.
POST	`/api/campaign/start`	signed in	`leads/server/campaign.js:94`	

`/api/comments`

METHOD	PATH	GUARD	SOURCE
PATCH	`/api/comments/:id`	signed in *(inherited)*	`leads/server/leads.js:481`

METHOD	PATH	GUARD	SOURCE
DELETE	`/api/comments/:id`	signed in *(inherited)*	`leads/server/leads.js:497`

`/api/counts`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/counts`	signed in	`leads/server/counts.js:19`	
GET	`/api/counts`	signed in	`leads/server/demo.js:342`	Counted from the same rows the list serves.

`/api/cron`

METHOD	PATH	GUARD	SOURCE	
POST	`/api/cron/daily`	—	`leads/server/digest.js:118`	The knock. Not behind requireUser — the caller is a scheduler, not a person — so it carries a...

`/api/dnc-import`

METHOD	PATH	GUARD	SOURCE
POST	`/api/dnc-import`	service token (imports:write) or admin	`leads/server/imports.js:374`

`/api/followups`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/followups`	signed in	`leads/server/followups.js:128`	Everything outstanding for the caller — the My tasks card.
POST	`/api/followups/:id/done`	signed in	`leads/server/followups.js:146`	

`/api/identity-lookup`

METHOD	PATH	GUARD	SOURCE
GET	`/api/identity-lookup`	service token (imports:write) or admin	`leads/server/imports.js:357`

`/api/import-jobs`

METHOD	PATH	GUARD	SOURCE
POST	`/api/import-jobs`	service token (imports:write) or admin	`leads/server/imports.js:79`
GET	`/api/import-jobs/:id`	service token (imports:write) or admin	`leads/server/imports.js:186`
POST	`/api/import-jobs/:id/abort`	service token (imports:write) or admin	`leads/server/imports.js:191`

METHOD	PATH	GUARD	SOURCE
POST	`/api/import-jobs/:id/commit`	service token (imports:write) or admin	`leads/server/imports.js:174`
POST	`/api/import-jobs/:id/records`	service token (imports:write) or admin	`leads/server/imports.js:152`

`/api/job-search`

METHOD	PATH	GUARD	SOURCE
POST	`/api/job-search`	admin *(inherited)*	`leads/server/jobsearch.js:561`

`/api/leads`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/leads`	signed in	`leads/server/demo.js:266`	The whole list, not just the distribute query — otherwise the tabs sit empty while the KPI ti…
GET	`/api/leads`	signed in *(inherited)*	`leads/server/leads.js:163`	
PATCH	`/api/leads/:id`	own lead *(inherited)*	`leads/server/leads.js:276`	status / owner / favorite — the three judgement fields the UI edits
DELETE	`/api/leads/:id`	admin, own lead *(inherited)*	`leads/server/trash.js:110`	
POST	`/api/leads/:id/call`	signed in, own lead *(inherited)*	`leads/server/calls.js:98`	POST /api/leads/:id/call — the softphone reporting how a call went.
GET	`/api/leads/:id/comments`	own lead *(inherited)*	`leads/server/leads.js:442`	
POST	`/api/leads/:id/comments`	own lead *(inherited)*	`leads/server/leads.js:453`	
GET	`/api/leads/:id/delete-preview`	admin, own lead *(inherited)*	`leads/server/trash.js:94`	what the confirm modal shows before the click:
GET	`/api/leads/:id/followup`	signed in, own lead *(inherited)*	`leads/server/followups.js:104`	What is set on this lead right now — the reading pane asks on every open.
POST	`/api/leads/:id/followup`	signed in, own lead *(inherited)*	`leads/server/followups.js:51`	Set (or replace) the reminder on a lead.

METHOD	PATH	GUARD	SOURCE	
DELETE	`/api/leads/:id/followup`	signed in, own lead *(inherited)*	`leads/server/followups.js:116`	
POST	`/api/leads/:id/remove`	own lead *(inherited)*	`leads/server/leads.js:339`	removal is a phone ban:
GET	`/api/leads/:id/remove-preview`	own lead *(inherited)*	`leads/server/leads.js:323`	how many leads a ban would take down — the modal's blast radius line
GET	`/api/leads/:id/research`	signed in, own lead *(inherited)*	`leads/server/research.js:49`	GET — whatever we already hold.
POST	`/api/leads/:id/research`	signed in, own lead *(inherited)*	`leads/server/research.js:90`	POST — buy a briefing.
POST	`/api/leads/:id/restore`	own lead *(inherited)*	`leads/server/leads.js:410`	
GET	`/api/leads/:key`	signed in	`leads/server/demo.js:310`	Opening a lead.
GET	`/api/leads/:key`	signed in *(inherited)*	`leads/server/leads.js:202`	one lead plus everything the reading pane shows beside it

`/api/me`

METHOD	PATH	GUARD	SOURCE
GET	`/api/me`	signed in	`leads/server/index.js:74`

`/api/owners`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/owners`	signed in	`leads/server/bulk.js:211`	Two populations, one list:

`/api/phone-refresh`

METHOD	PATH	GUARD	SOURCE
POST	`/api/phone-refresh`	admin *(inherited)*	`leads/server/webphone.js:125`

`/api/recents`

METHOD	PATH	GUARD	SOURCE
GET	`/api/recents`	signed in *(inherited)*	`leads/server/recents.js:20`
POST	`/api/recents`	signed in *(inherited)*	`leads/server/recents.js:38`
DELETE	`/api/recents`	signed in *(inherited)*	`leads/server/recents.js:63`

`/api/research`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/research/recent`	signed in	`leads/server/research.js:65`	GET /api/research/recent — everything researched lately, for the sidebar's Research section.

`/api/tasks`

METHOD	PATH	GUARD	SOURCE
GET	`/api/tasks`	signed in	`leads/server/tasks.js:32`

`/api/trash`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/trash`	admin	`leads/server/trash.js:142`	
DELETE	`/api/trash`	owner	`leads/server/trash.js:191`	empty it now — this is the destructive one, and the only action in the app with no way back.
POST	`/api/trash/:id/restore`	admin	`leads/server/trash.js:156`	

`/api/users`

METHOD	PATH	GUARD	SOURCE
GET	`/api/users`	admin *(inherited)*	`leads/server/users.js:49`
POST	`/api/users`	admin *(inherited)*	`leads/server/users.js:67`
PATCH	`/api/users/:id`	admin *(inherited)*	`leads/server/users.js:118`

`/api/voice`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/voice/config`	signed in	`leads/server/voice.js:159`	
POST	`/api/voice/token`	signed in	`leads/server/voice.js:184`	The SDK's oAuthCallBack fetches this.

Karma Connect — 62 routes

`/api/admin`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/admin/activity`	admin, signed in	`server.js:1113`	——————————————————— Team activity ——————————————————— Admin-only:

METHOD	PATH	GUARD	SOURCE	
GET	`/api/admin/users`	admin, signed in	`server.js:1044`	———————————————————— Admin:
DELETE	`/api/admin/users/:id`	owner, signed in	`server.js:1086`	Remove a member (cascades their data).
POST	`/api/admin/users/:id/role`	owner, signed in	`server.js:1064`	Owner only. An admin who can appoint admins is an owner in all but name, and one compromised...

`/api/admins`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/admins`	signed in	`server.js:1102`	Any signed-in user can see the list of admins (to choose who should approve a lead).

`/api/analytics`

METHOD	PATH	GUARD	SOURCE
GET	`/api/analytics`	signed in	`server.js:681`

`/api/approvals`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/approvals`	admin, signed in	`server.js:1002`	Admin: list approval requests (pending first).
POST	`/api/approvals`	signed in	`server.js:951`	———————————————————— Lead approvals ———————————————————— Member requests approval...
POST	`/api/approvals/:id/decide`	admin, signed in	`server.js:1016`	Admin: approve or deny a request.
GET	`/api/approvals/mine`	signed in	`server.js:994`	Member: their own lead statuses, to gate the UI.

`/api/auth`

METHOD	PATH	GUARD	SOURCE	
DELETE	`/api/auth/account`	signed in	`server.js:394`	Delete the signed-in user's own account (password-confirmed).
POST	`/api/auth/forgot`	rate-limited	`server.js:308`	Request a password reset.
GET	`/api/auth/me`	signed in	`server.js:377`	

METHOD	PATH	GUARD	SOURCE	
POST	`/api/auth/password`	signed in, rate-limited	`server.js:355`	Change password while signed in (current password required).
POST	`/api/auth/promote`	signed in, rate-limited	`server.js:349`	Self-promotion is gone — see the note in /api/auth/signup.
POST	`/api/auth/resend`	rate-limited	`server.js:290`	Re-send the confirmation email for an unverified account.
POST	`/api/auth/reset`	rate-limited	`server.js:326`	Complete a password reset:
POST	`/api/auth/signin`	rate-limited	`server.js:250`	
POST	`/api/auth/signout`	—	`server.js:372`	
POST	`/api/auth/signup`	rate-limited	`server.js:179`	
GET	`/api/auth/verify`	—	`server.js:273`	Click target from the confirmation email.

`/api/campaigns`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/campaigns`	signed in	`server.js:760`	List campaigns with step + enrollment counts.
POST	`/api/campaigns`	signed in	`server.js:775`	Create a campaign with its steps.
GET	`/api/campaigns/:id`	signed in	`server.js:799`	One campaign: steps + enrollments (joined to prospect label/status).
PATCH	`/api/campaigns/:id`	signed in	`server.js:816`	Update name / sender / status (active|archived).
DELETE	`/api/campaigns/:id`	signed in	`server.js:847`	
POST	`/api/campaigns/:id/enroll`	signed in	`server.js:855`	Enroll leads. Body: { prospectIds: [...] }. Sets each to step 0, due after step 0's delay.
PUT	`/api/campaigns/:id/steps`	signed in	`server.js:831`	Replace a campaign's steps wholesale.

`/api/claude`

METHOD	PATH	GUARD	SOURCE	
POST	`/api/claude`	signed in	`server.js:1275`	——————————————————— Anthropic proxy ——————————————————— In dev the proxy is open...

`/api/coach`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/coach/changes`	signed in	`server.js:671`	The change log:

METHOD	PATH	GUARD	SOURCE	
GET	`/api/coach/dataset`	signed in	`server.js:616`	Coach dataset:
GET	`/api/coach/playbook`	signed in	`server.js:635`	Read all learned playbooks for the user (keyed by kind).
PUT	`/api/coach/playbook`	signed in	`server.js:646`	Save/replace a learned playbook for one kind.

`/api/connect`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/connect/leads/:id/notes`	signed in	`server.js:557`	Lead notes & activity
POST	`/api/connect/leads/:id/notes`	signed in	`server.js:567`	
DELETE	`/api/connect/leads/:id/notes/:noteId`	signed in	`server.js:579`	
GET	`/api/connect/leads/all`	signed in	`server.js:479`	All of the user's leads (minimal fields) for the board + export.
POST	`/api/connect/leads/brief.pdf`	signed in	`server.js:522`	One prospect's brief PDF (details + computed fit + AI profile).
GET	`/api/connect/leads/export.pdf`	signed in	`server.js:509`	Branded PDF export of the user's leads (rendering in server/pdf.cjs).

`/api/enrollments`

METHOD	PATH	GUARD	SOURCE	
POST	`/api/enrollments/:id/advance`	signed in	`server.js:885`	Advance one enrollment to its next step (called when the current step is marked sent).
POST	`/api/enrollments/:id/stop`	signed in	`server.js:913`	Stop an enrollment (remove the lead from the cadence).

`/api/health`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/health`	—	`server.js:1293`	Health check for Render.

`/api/history`

METHOD	PATH	GUARD	SOURCE
GET	`/api/history`	signed in	`server.js:413`

METHOD	PATH	GUARD	SOURCE	
POST	`/api/history`	signed in	`server.js:450`	Upsert one or more entries.
DELETE	`/api/history/:id`	signed in	`server.js:471`	
POST	`/api/history/:id/reminder`	signed in	`server.js:531`	Set or clear a lead's follow-up reminder (next-action date + optional note).
POST	`/api/history/:id/status`	signed in	`server.js:432`	Update just the pipeline status of one lead.

`/api/message-sends`

METHOD	PATH	GUARD	SOURCE	
POST	`/api/message-sends`	signed in	`server.js:594`	Log that a generated message was sent (the A/B event).

`/api/outreach`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/outreach/agent/download`	signed in	`server.js:1220`	The agent's source, zipped, so a recruiter can run it without GitHub access or a clone.
POST	`/api/outreach/agent/enable`	signed in	`server.js:1234`	The kill switch.
POST	`/api/outreach/agent/lease`	signed in	`server.js:1244`	Hand the worker its next batch.
POST	`/api/outreach/agent/report/:id`	signed in	`server.js:1258`	Report one leased item back.
GET	`/api/outreach/agent/status`	signed in	`server.js:1211`	Agent state for the Send Queue strip:
GET	`/api/outreach/queue`	signed in	`server.js:1156`	List this user's queue (optional ?status= and ?channel= filters).
POST	`/api/outreach/queue`	signed in	`server.js:1147`	———————————————— Outreach send queue ———————————————— Queue a generated messag…
POST	`/api/outreach/queue/:id/approve`	signed in	`server.js:1164`	Approve a pending item (schedules it; email then auto-sends on the next tick).
POST	`/api/outreach/queue/:id/send-now`	signed in	`server.js:1177`	Send an approved email item immediately (don't wait for the tick).
POST	`/api/outreach/queue/:id/sent`	signed in	`server.js:1187`	Mark a LinkedIn item sent after the human one-click hand-off.
POST	`/api/outreach/queue/:id/skip`	signed in	`server.js:1196`	Skip / cancel a queued item.

`/api/reminders`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/reminders`	signed in	`server.js:546`	All leads with a follow-up date, soonest first — powers the Reminders view + badge.

`/api/stats`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/stats`	signed in	`server.js:490`	Pipeline metrics for the dashboard (counts per stage + this week).

`/api/usage`

METHOD	PATH	GUARD	SOURCE	
GET	`/api/usage`	signed in	`server.js:923`	——————————————————————— Usage ———————————————————————

What this cannot tell you

The generator reads route declarations, not behaviour. It does not know:

- **what a route returns.** Read the handler.
- **routes built from computed paths.** None exist today; if one is added it will be missing here silently. The total above is the check.
- **whether a guard is sufficient.** `signed in` on a route that mutates other people's data would look correct in this table and be wrong.
- **client-side gating.** `.admin-only` in the CSS is presentation. The server is the only thing that enforces anything.
- **which branch actually registers.** `/api/auth/login`, `/api/auth/callback` and `/api/auth/logout` each appear twice from `leads/server/auth.js`. They are not duplicates: the second set is the `else` branch that renders "Sign-in is not configured" when the WorkOS variables are missing. Only one set is ever mounted.

Routes that are deliberately open

Fifteen routes are reachable without a session — **ten with no guard at all, five behind a rate limiter only** — and none of them should be otherwise:

ROUTE	WHY
`POST /api/auth/signup`, `signin`, `forgot`, `reset`, `resend`	you cannot require a session to get a session. These are the five that are rate-limited
`GET /api/auth/verify`	the link in a verification email

ROUTE	WHY
`POST /api/auth/signout`, `POST /api/auth/logout`	clearing a cookie must work even with a bad one
`GET /api/auth/login`, `GET /api/auth/callback`	the sign-in round trip
`GET /api/health`	the Render health check, which has no cookie
`POST /api/cron/daily`	the caller is a scheduler, not a person. It carries the `CRON_TOKEN` shared secret instead, compared in full, and **returns 503 when the token is unset rather than defaulting to open**

If a future route appears in this list, that is the thing to look at first.

6 Security

What is enforced, where, and the gaps

What is enforced, where, and why it is that way. Read alongside API, which lists the guard on every one of the 133 routes.

The threat model is honest and small: **eleven employees of one company, behind a sign-in, holding 36,619 commercial contact records and the keys to a phone system.** There is no anonymous public surface, no customer data, no payments. The realistic risks are, in order:

1. A credential or token leaking — the Zoho and Apify keys are durable and cost money.
2. A member reaching leads that are not theirs, or promoting themselves.
3. An irreversible action taken by someone who should not have it — emptying the bin, deleting an account.

Everything below is aimed at those three.

Sign-in

One session for both products: an **HS256 JWT in an httpOnly cookie**.



PROPERTY	VALUE	WHY
Cookie	`kc_session`	
`HttpOnly`	yes	page scripts cannot read it — the localStorage token it replaced could not claim that
`SameSite`	`Lax`	off cross-site requests, but still arrives when someone follows a link in
`Path`	`/`	one cookie covers Connect at `/` and Leads at `/leads`
`Secure`	production only	a `Secure` cookie on `http://localhost` is silently never stored, which would make local sign-in impossible. Gated on `RENDER` or `NODE_ENV=production`
TTL	30 days	
Algorithm	HS256, `AUTH_JWT_SECRET`	the same helpers the AI proxy uses, so there is **one** signing secret rather than two schemes to keep in step

Why a cookie and not a bearer token. Karma Leads' front end never sets an `Authorization` header — it is vanilla JS that relies on the browser attaching a cookie by itself. A token in `localStorage` could not be

shared with it without rewriting every call it makes. The cookie is shared for free.

Verification is done properly

`verifyJWT` in `api/_proxy.cjs`:

- recomputes the HMAC and compares with `crypto.timingSafeEqual`, after a length check
- rejects an expired `exp`
- rejects a mismatched `iss` when one is expected
- returns `null` rather than throwing — a malformed cookie is "not signed in", exactly like no cookie

Revocation

There is no session store, so there is no per-session revoke. **Rotating `AUTH_JWT_SECRET` signs everybody out of both halves.** That is the lever; it is blunt and it works. Use it if a laptop is lost.

Password handling (Connect only)

`bcryptjs`, cost 12. Note `DUMMY_HASH` in `server/auth.cjs`: a real bcrypt hash compared against when the email does not exist, so a **missing account takes the same time as a wrong password**. Without it, response timing enumerates who has an account.

WorkOS

Still accepted as a fallback so sessions issued before the cutover are not thrown out mid-afternoon. Nobody signs in through it now. It was replaced because it had been set up by someone who left and nobody remaining could administer it — allowlisting a callback URL was impossible. **An identity provider only one departed person can operate is a single point of failure, not a security control.**

Authorisation: a rank ladder, not a set of labels

```
const RANK = { member: 0, admin: 1, owner: 2 };
const rankOf = (role) ⇒ RANK[role] ?? 0;
```
JS

Every check is **"at least"**, never equality:

```
if (rankOf(req.user.role) < RANK.admin) return res.status(403)…
```
JS

This matters and it is a rule with teeth. `role === "admin"` locks the *owner* out of admin screens, because an owner is not an admin — they outrank one. An unknown role falls to `0`, so a typo in the database demotes rather than escalates.

TIER	COUNT TODAY	WHAT IT IS FOR
`member`	7	works the leads assigned to them
`admin`	3	runs the floor. Everything an admin can do is recoverable or merely costs money
`owner`	1	the irreversible things

The owner tier lives in the environment

```JS
const OWNER_EMAILS = (process.env.OWNER_EMAILS ?? "tenzinwoesel@karmastaff.com")
const isOwnerEmail = (email) ⇒ OWNER_EMAILS.includes(…)
```

`OWNER_EMAILS` **wins over the stored role, never the other way round.** So:

- write access to `organization_memberships` does not grant the irreversible powers — you also need the Render dashboard
- the tier cannot be lost by a bad migration or an accidental `UPDATE`
- changing hands is an environment change, which is logged and deliberate

See ADR-002.

What only an owner can do

Three routes, and each is chosen on the same test — *can this be undone?*

ROUTE	WHY OWNER
`DELETE /api/trash`	empties the bin. **The only action in the app with no way back**
`POST /api/admin/users/:id/role`	appoints admins. An admin who can appoint admins is an owner in all but name, and one compromised account would be enough
`DELETE /api/admin/users/:id`	removes a member and cascades their data

Admin is not self-served

There is no `ADMIN_SIGNUP_CODE`. It was a single shared string that turned whoever typed it into an admin: it could not be rotated without locking out the people who already had it, and it could not be audited, because a shared secret has no subject. An existing owner promotes people now.

Row-level access: members see only their own leads

`requireOwnLead` in `leads/server/leads.js`, mounted with `router.use("/api/leads/:id", requireOwnLead)` so it covers every sub-route at once rather than being repeated and eventually forgotten:

```JS
if (rankOf(req.user.role) ⩾ RANK.admin) return next();   // admins see all
const row = await query("SELECT owner FROM leads WHERE id = $1 AND deleted_at IS NULL"…);
if (!row) return res.status(404).json({ error: "no such lead" });
if (row.owner.trim() ≠≠ req.user.email.trim())
  return res.status(404).json({ error: "no such lead" });
```

404, not 403, and that is deliberate. A 403 confirms the lead exists. A member probing ids would learn the shape of the base — how many leads there are, which ids are live — from the difference between the two codes. Both answers are "no such lead".

Two caveats worth knowing:

- The comparison is on `leads.owner`, **which is free text**, matched by trimmed equality on the email. It is not a foreign key. Assignment goes through `GET /api/owners` so one person cannot become three owner strings via a typo, and an account's assignable value is its **email**, never its display name.
- Because it is free text, a lead can be owned by an address with no account — and 2,011 currently are. Those are unreachable by every member, which is safe but not intended. See The data.

How the 133 routes are actually guarded

Counted from API, which is generated from the route declarations:

GUARD	ROUTES
signed in	71
signed in + rate-limited	2
signed in + own lead	12
admin *(admins bypass the own-lead check)*	23
owner	3
service token *(imports:write)* or admin	7
rate-limited only, no session	5
no guard at all	10
total	**133**

So **fifteen routes are reachable without a session**: ten with no guard, five behind a rate limiter only. All fifteen are correct — the sign-in flows (you cannot require a session in order to get one), `/api/health` for Render's checker, and `/api/cron/daily`. API.md lists them with reasons; a *new* entry in that list is the thing to look at.

`POST /api/cron/daily` is the interesting one. Its caller is a scheduler, not a person, so it carries a shared secret in `x-cron-token`, compared in full against `CRON_TOKEN`, and **returns 503 when `CRON_TOKEN` is unset rather than defaulting to open**. Failing closed on a missing config value is the right default and is not free — it has to be written that way.

The client enforces nothing

`.admin-only` in `app.css` hides buttons. That is presentation. Every check above is server-side, and the API returns the same answer whether the button was visible or not. This was **verified by a real probe across all three tiers**, not assumed.

Machine callers

The Python import pipeline is not a user. It authenticates with a token from `service_tokens`:

- only the **SHA-256 hash** is stored (`token_hash`, unique); the token itself is shown once at mint

- tokens carry a `scopes` **array**, checked per route — `imports:write` for the seven ingest routes
- `revoked_at` kills one without touching the others
- `requireScope(scope, requireAdmin)` falls through to the admin-user path when the caller is not a service, so the same route serves both a human admin and the pipeline
- the actor is recorded as `service:<name>` in `activity_log`, so machine writes are distinguishable from human ones in the feed

The pipeline reaches the API over HTTP (`KARMA_API_URL` / `KARMA_API_TOKEN`) and **never connects to Postgres directly**, so every ingest passes the same validation and dedupe a human upload does.

HTTP headers

Set for every response in `server.js`:

HEADER	VALUE
`X-Content-Type-Options`	`nosniff`
`X-Frame-Options`	`DENY`
`Referrer-Policy`	`strict-origin-when-cross-origin`
`Strict-Transport-Security`	`max-age=31536000; includeSubDomains` *(production only)*
`Permissions-Policy`	`camera=(), microphone=(), geolocation=()` — **except on** `/leads`, where it is `microphone=(self)`

The CSP is scoped per route, on purpose

Two policies. Karma Connect and everything else get the strict one:

```
default-src 'self'; base-uri 'self'; object-src 'none'; frame-ancestors 'none';
img-src 'self' data:; script-src 'self';
style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
font-src 'self' https://fonts.gstatic.com data:; connect-src 'self'
```

`/leads` gets a widened one, because Zoho's click-to-call is a **WebRTC softphone that runs inside the page**: its SDK is served from `zohostatic.com` and it talks to Zoho over HTTPS and WebSocket, none of which the strict policy permits.

The widening is scoped to Zoho's host family — `zoho.com`, `zohoapis.com`, `zohostatic.com`, `zohocdn.com`, `zohopublic.com`, `zohovoice.com`, plus `wss:` for signalling — rather than opened to the web. Wildcards are used for `connect-src` because the signalling and TURN hosts are regional (`us-relay2.zohovoice.com` and friends) and are not a published fixed list.

Widening it globally would hand every page in both products the right to load and talk to a third party, for the sake of one feature on one route. The per-route split costs one `req.path` test.

Similarly, the microphone is granted only where a call can happen. Camera and geolocation are denied everywhere, including `/leads`.

A TRAP THAT HAS ALREADY COST TIME

Development serves no CSP at all (the whole block is behind `if (!DEV)`), because Vite serves the HTML in dev. So a CSP problem is invisible locally and appears only on the deploy — and the Zoho SDK reports a blocked request as `Unauthorized access`. **A blocked request and a refused credential look identical from inside the SDK.** If Zoho works locally and fails deployed, suspect the policy before the token.

The AI proxy

`api/_proxy.cjs` sits between the browser and Anthropic. **The browser is not trusted with the request body** — `sanitizeBody()` rebuilds it:

CONTROL	DEFAULT	EFFECT
`ALLOWED_MODELS`	`claude-sonnet-4-6,claude-haiku-4-5`	a model not on the list is rejected, so the browser cannot ask for an expensive one
`MAX_TOKENS_CAP`	8000	`max_tokens` is clamped into range, not trusted
`MAX_TOOLS`	2	the tool array is truncated
`MAX_SEARCH_USES`	4	each web-search tool's `max_uses` is clamped
`RATE_LIMIT_RPM`	—	per-caller request rate
`DAILY_TOKEN_BUDGET`	—	spend ceiling
`BREAKER_FAILS` / `BREAKER_COOLDOWN`	—	circuit breaker, so a failing upstream is not hammered
`RESPONSE_CACHE_TTL`	—	identical requests are served from cache

Cost is recorded per call and rolled up into `usage_daily`, so AI spend is visible per person per day rather than arriving as one monthly surprise.

`ANTHROPIC_API_KEY` never reaches the browser — that is the proxy's first purpose. The clamps are its second: without them, anyone with a session could run up the bill from the console.

> Keep `ALLOWED_MODELS` current when models change. The allowlist is the control, and a stale list fails closed — features stop rather than overspend — which is the right direction but still an outage.

Sign-in rate limits

ENDPOINT	WINDOW	MAX	KEYED ON
`signin`	15 min	12	IP + email
`signup`	60 min	8	IP
`resend`	60 min	6	IP + email
`forgot`	60 min	6	IP + email
`reset`	60 min	15	IP

ENDPOINT	WINDOW	MAX	KEYED ON
`promote`	15 min	8	user ID
`password`	15 min	10	user ID

Keying sign-in on **IP + email** rather than IP alone means one person fumbling their password does not lock out a colleague behind the same office NAT, while still stopping a single address being sprayed.

Without `UPSTASH_REDIS_REST_URL` / `_TOKEN` these counters are per-instance. With one instance that is the same thing; it stops being true the moment the service scales out.

Secrets

- `.env` is gitignored. **Nothing in this project's secrets is low-value** — they are durable keys to a phone system, a scraping account and a metered AI API.
- Never paste a placeholder into a config value. `YOUR_CLIENT_ID`, `<from the other service>` and a Zoho **grant code** have each reached production here.
- A Zoho grant code is not a refresh token. Pasting the former produces `invalid_code`, and the error's `refreshTokenPrefix` is how that was identified.
- `DEV_AUTH_EMAIL` / `DEV_AUTH_ROLE` bypass sign-in entirely and print a banner saying so at boot. **Never set on a deploy.**

A grep trap, now fixed

`api/_proxy.cjs` contained two raw NUL bytes — a deliberate delimiter in a cache-key hash, but written as literal bytes. `grep` therefore classified the file as binary and **skipped it**, so a repo-wide search silently missed 17 environment variables that only this file reads, including `ANTHROPIC_API_KEY` and `UPSTASH_REDIS_REST_TOKEN`.

Replaced with `\u0000` escapes — identical to JavaScript, greppable to everything else. Worth remembering if a secret sweep over this repo ever comes back suspiciously clean: check for `grep -a`.

Known gaps

Stated rather than glossed.

GAP	SEVERITY	NOTES
2,011 leads owned by an accountless address	medium	not an exposure — they are unreachable by everyone — but it is unmanaged data. See DATABASE.md
No per-session revocation	low	rotating `AUTH_JWT_SECRET` signs out everybody. Acceptable at eleven users
Two user tables	medium	a person removed from one may remain in the other. Deprovisioning is a two-step that nothing enforces
The old service is still live	medium	`karma-leads-nocodb` serves frozen code against the *live* database, so old authorisation logic is still reachable. The strongest argument for suspending it

GAP	SEVERITY	NOTES
Rate limits are per-instance without Redis	low	true today, false the moment it scales
No audit of reads	low	`activity_log` records `open`, but a member enumerating their own leads is not distinguishable from working
Playwright role probes are not in CI	low	the three-tier matrix was verified by hand; nothing re-checks it on a change

The last one is the one to fix first, because it is the only gap where a *regression* would be silent. Everything else on this list is known and stable.

7 External services

Ten dependencies and how each one fails

Ten dependencies. For each: what it is for, what it costs, how it fails, and what the app does when it does.

The governing principle is in The product: **never make the rep the fallback.** When something outside fails, the app degrades to something usable and says so, rather than showing an error and stopping.

SERVICE	FOR	COST	KEY NEEDED	FAILURE IS
Anthropic	prospect briefings, outreach drafting	per token	`ANTHROPIC_API_KEY`	visible, feature stops
Zoho Voice	click-to-call	per seat	OAuth refresh token	**degraded, not fatal**
Apify	job-board and LinkedIn scraping	per run	`APIFY_TOKEN`	feature disabled
Mail gateway	every email the app sends	internal	`GATEWAY_TOKEN`	**silent in three places**
NWS `api.weather.gov`	active weather alerts	free	none	named as down
FEMA OpenFEMA	declared disasters	free	none	named as down
NIFC ArcGIS	active wildfires	free	none	named as down
USGS	earthquakes	free	none	named as down
NOAA NHC	active tropical storms	free	none	named as down
LinkedIn	company and profile research	via Apify	actor id	empty briefing fields

Anthropic

Reached only through `api/_proxy.cjs`, never from the browser. The proxy exists so the API key never reaches a client, and stays because it is also where spend is controlled.

It **rebuilds the request** rather than forwarding it — `sanitizeBody()` — so the browser cannot ask for a model it likes or a token count it invents. The full clamp table is in Security.

BEHAVIOUR	NOTES
Rate limit	`RATE_LIMIT_RPM`, per caller
Daily budget	`DAILY_TOKEN_BUDGET`
Circuit breaker	`BREAKER_FAILS` failures opens it for `BREAKER_COOLDOWN`
Response cache	`RESPONSE_CACHE_TTL`; keyed on model + system + user text, **never** for tool-using calls, because a live-search answer must not be replayed
Cost recording	every call, rolled into `usage_daily` per user per day

BEHAVIOUR	NOTES
Shared state	`UPSTASH_REDIS_*`. Without it the limits are per-instance

Where it is used. Prospect briefings (`leads/server/brief.js` builds the prompt and parses the reply — both are unit-tested without a network) and Connect's outreach drafting and coach.

How it fails. Loudly and locally: the feature returns an error and the rest of the app is unaffected. The breaker means a broken upstream is not hammered.

> `ALLOWED_MODELS` defaults to `claude-sonnet-4-6,claude-haiku-4-5`. The allowlist is the control, so it needs updating when the models in use change — a stale list fails closed, which stops the feature rather than overspending. Right direction, still an outage.

The briefing's one rule

An empty field is a correct answer. A fabricated one loses the call in ten seconds, and the rep will not find out until they are on the phone. Every briefing carries a confidence rating, and the prompt is built to prefer silence over invention.

Zoho Voice

Read this before changing any Zoho code: the current blocker is a licence tier, not code. Zoho *Telephony* agents are denied the Voice softphone. No token, scope, or configuration change fixes that.

How it is meant to work

A WebRTC softphone that runs **inside the page**, which is why `/leads` gets its own widened Content-Security-Policy and is the only route granted `microphone=(self)`. See Security.

Auth is an OAuth Self Client: `ZOHO_CLIENT_ID` + `ZOHO_CLIENT_SECRET` + `ZOHO_REFRESH_TOKEN`, exchanged for short-lived access tokens. `GET /api/voice/config` reports **which scope was actually granted** — the one question the Zoho dashboard will not answer.

How it degrades

The whole hand-off is one function, `dialHandOff()`:

1. If `ZOHO_DIAL_URL` contains `{number}`, open it with the number substituted.
2. Otherwise **copy the number to the clipboard** and say so.

The number is copied in *every* path, including the successful one, because a rep who can see the digits is never stuck. `ZOHO_DIAL_URL` should be **empty** in production — copy-only is the working configuration today.

What has actually gone wrong

Every row here was a real incident, and only one was a code bug in the integration itself.

SYMPTOM	CAUSE
`Invalid OAuth scope`	the token was minted asking `ZohoVoice.sdk.READ`, by our own helper script. Now `.ALL`
every token read as read-only	a literal `0x08` byte inside the scope regex — a shell heredoc ate a backslash, turning `\b` into a control character. Found with `od -c`
`invalid_code`	a **grant code** was pasted into `ZOHO_REFRESH_TOKEN`. The error's `refreshTokenPrefix` identified it
`Access Denied` on the dialler	our own fallback opened `voice.zoho.com` — the exact page a Telephony licence denies
Windows app-chooser with no Zoho in it	`ZOHO_DIAL_URL=tel:{number}`; Zoho is not a registered `tel:` handler
blanking `ZOHO_DIAL_URL` changed nothing	`	`treated empty as unset. Now` === undefined`
"Unauthorized access" on deploy, fine locally	**the CSP**. Development serves no CSP at all, so a host blocked in production was never blocked locally — and the SDK reports a blocked request identically to a refused credential

The lesson that generalises: *the SDK's error message describes what it observed, not what went wrong.*

Apify

Two uses, both admin-initiated and both paid per run:

USE	ACTOR	ROUTE
Job-board scraping — finds firms that are hiring, i.e. short-handed	built in	`POST /api/job-search` (admin)
LinkedIn company / profile research	`APIFY_LINKEDIN_COMPANY_ACTOR`, `APIFY_LINKEDIN_PROFILE_ACTOR`	the briefing

`GET /api/apify-usage` reports spend. Because runs cost money, nothing triggers them automatically — a person presses a button.

Without `APIFY_TOKEN` these features are disabled, not broken. The server says "Apify token not configured — set the APIFY_TOKEN environment variable" and everything else works. Currently unset, so job search and LinkedIn research are off.

`leads/server/linkedin.js` normalises actor output shapes and is unit-tested without a network, because **actors change their output format without warning** — that is the integration's main long-term risk, and the tests are the early warning.

On the LinkedIn scraping

Used in a deliberately limited way, at the user's stated intent. It is research-before-a-call, one prospect at a time, not bulk harvesting.

The mail gateway

One way out. Every email the app sends POSTs to `${GATEWAY_URL}/send` with a bearer token. Not SMTP.

Eight message types: verification, password reset, approval request, approval decision, outreach, follow-up reminder, storm digest, team invite.

Good things about it:

- **Idempotency keys** are supported and used where a duplicate would be embarrassing — the digest sends `storm-digest-<date>-<recipient>`, so a double knock cannot mail the team twice.
- **Errors are mapped to causes**, not passed through: `401` says check `GATEWAY_TOKEN`; `403` says the admin must allowlist `MAIL_FROM`; `429` reports the `Retry-After`. That saves a hunt through the gateway's own logs.

Its failure mode is the worst in the system

`emailVerifyOn` is just `GATEWAY_URL && GATEWAY_TOKEN`. When it is false, **three features fail in three different-looking ways**:

WHAT A USER SEES	WHAT ACTUALLY HAPPENED
"The invite didn't send"	no gateway
"My follow-back reminder never arrived"	no gateway
digest reports `skipped: "mail is not configured"`	no gateway

One cause, three bug reports, none of which mentions email configuration. If any one of these is reported, check `GATEWAY_URL` and `GATEWAY_TOKEN` **first** — see RUNBOOK.

The five public data feeds

Storm watch. All five are official, free, and need no key — which is most of why this feature is worth having.

FEED	URL	FILTER
NWS active alerts	`api.weather.gov/alerts/active`	the ~20% that mean property damage
FEMA disaster declarations	`fema.gov/api/open/v2/DisasterDeclarationsSummaries`	last **45 days**
NIFC active wildfires	`services3.arcgis.com/…`	≥ **100 acres**
USGS earthquakes	`earthquake.usgs.gov/…/2.5_day.geojson`	≥ **magnitude 4.0**
NOAA NHC storms	`nhc.noaa.gov/CurrentStorms.json`	all active

Cache 5 minutes, request timeout 20 seconds, `User-Agent` from `NWS_USER_AGENT` (the weather service asks for a contact address and may throttle without one).

The thresholds are editorial, and that is the product

Each filter encodes a judgement about what a restoration company can sell into:

- **100 acres** — smaller than that is a local crew's day, not a staffing shortage.
- **Magnitude 4.0** — below it, nothing needs restoring.
- **45 days** for FEMA — a declaration stays interesting while cleanup runs, which is months, but as a "what just happened" list six weeks is the useful window.

Weather alerts are tiered rather than filtered:

TIER	MEANING	EXAMPLES
3	property is being destroyed right now	tornado, hurricane, storm surge, flash flood, ice storm, blizzard, extreme wind
2	damage likely, crews will be needed	tropical storm, severe thunderstorm, winter storm, high wind, coastal flood, flood, fire warning, red flag, freezing rain
1	worth watching	winter weather, wind advisory, fire weather, hard freeze, dense smoke

Matched on the event name *containing* the key, because the NWS emits dozens of variants — "Tornado Warning", "Tornado Watch", "PDS Tornado Warning" — and a closed list would miss the one that matters. **Order counts: first match wins**, so specific sits above general ("flash flood" before "flood").

`NOT_A_STATE` drops marine and coastal-leisure UGC zones (`AM`, `AN`, `GM`, `LC`, `LE`, `LH`, `LM`, `LO`, `LS`, `PH`, `PK`, `PM`, `PS`, `PZ`, `SL`). A gale warning on Lake Erie is not restoration work.

Failing one feed does not fail the feature

```
const [f, w, q, s] = await Promise.allSettled([fema(), fires(), quakes(), storms()]);
…
down: [["FEMA", f], ["NIFC", w], ["USGS", q], ["NHC", s]]
        .filter(([, r]) ⇒ r.status ≡≡ "rejected").map(([n]) ⇒ n),
```

JS

`Promise.allSettled` throughout, and **the response carries a `down` array that the UI displays.** A partial map that names what is missing is honest; a blank page or a stale map presented as current is not. This is the product principle — *say what is not known* — implemented in eight lines.

What joins it to the product

The feeds answer "where did property just get damaged". The **join to the lead base** answers "and who do we already have on file there", and that is the part no generic weather app does. It is also where the data quality caveat bites: state matching has to handle both `FL` and `Florida`, and roughly a fifth of states in the base were *inferred* from city or area code at about 96% accuracy. Good enough to decide who to ring; not good enough to quote. See The data.

Render

The host, and two of its properties are load-bearing:

The free plan sleeps the process after ~15 minutes idle. Consequences:

- the first request after idle takes ~50 seconds — a timeout is not an outage
- `setInterval` **is not a scheduler.** At 7am nobody has touched the site since the night before, so anything in-process is not running. Scheduled work is triggered from outside by GitHub Actions (ADR-003)

Region matters for the database. `karmaleads_database` is in *virginia*, and Render's internal connection string only works from the same region. From elsewhere it must be the external URL — and getting it wrong looks like a hanging connection, not an auth error.

Also: `render.yaml` is stale and would deploy a broken service. See RUNBOOK.

GitHub Actions

Two workflows, both free.

`ci.yml` — push to `main` and every PR: `npm ci`, lint, test, build, and the API-docs drift check.

`daily-digest.yml` — knocks on `POST /api/cron/daily` at 11:00 and 12:00 UTC, because cron is UTC and has no idea about daylight saving, so one of the two is 7am Eastern. The server decides which; `job_runs` makes the other a no-op.

Two details worth copying:

- **It does not fail when it is merely unconfigured.** Missing secrets produce a GitHub *notice* and a clean exit. Mailing a red failure twice a day for "nobody has set this up yet" only teaches people to ignore the alerts that matter.
- **It distinguishes retryable from not.** `401` / `403` (wrong token) and `404` (wrong URL) stop immediately and say which; anything else retries three times 30 seconds apart, because it is most likely a free instance waking up.

Adding an integration

The bar, from the ones above:

1. **State plainly what happens when it is down.** If the answer is "the feature errors", say so in the UI. If it is "the rep is stuck", redesign it.
2. **Never let a paid call happen without a person asking.** Apify runs and briefings take a button. Free feeds may refresh themselves.
3. **Put the credential in the environment and nowhere else.** Twelve runtime dependencies is a feature; do not add an SDK to avoid writing a `fetch`.
4. **Normalise the response in one tested place.** `linkedin.js` exists because actors change shape without warning.
5. **Map errors to causes.** "Gateway 403" is a hunt; "the admin must allowlist MAIL_FROM" is a fix.

PART THREE · RUNNING IT

8 Runbook

What is set where, what breaks without it, how to check

What is set where, what breaks when it is not, and how to check — rather than how to guess. Most of the incidents in this project's history were a configuration value, and every one of them was one command away from being known.

The rule that matters: check the running system, not the source. `curl` the endpoint, query the database, look at the served HTML.

The estate

THING	WHERE	PLAN	NOTES
`karma-connect` web service	Render	free	the whole app, both halves
`karma-connect-db`	Render Postgres	free	Connect's data
`karmaleads_database`	Render Postgres	**paid**	36,619 leads, 114 MB, region **virginia**
`karma-leads-nocodb`	Render	—	⚠️ **the old service, still live**
Daily digest trigger	GitHub Actions	free	`.github/workflows/daily-digest.yml`
CI	GitHub Actions	free	`.github/workflows/ci.yml`
Mail	gateway via `GATEWAY_URL`	—	not SMTP
Telephony	Zoho	see §Zoho	licence-limited
Scrapers	Apify	pay per run	job search, LinkedIn
AI	Anthropic, via `api/_proxy.cjs`	pay per token	budgeted

⚠️ The old service is still running

`karma-leads-nocodb.onrender.com` serves **frozen code against the same live database**. This is not hypothetical: it is why reps reported that the new sections "weren't there" — they were bookmarked to the old host, which was serving `app.js?v=39`.

It is the single most likely cause of any future "it works for you but not for me". Before suspending it:

1. Confirm nothing points at it: check bookmarks with the team, and any saved links in email or chat.
2. Confirm it is not the thing holding a DNS record or a redirect.
3. Confirm the Python importer's `KARMA_API_URL` does **not** name it — it writes leads, and pointing at a frozen API is worse than pointing at none.
4. Suspend (do not delete) and leave it suspended for a fortnight.

```bash
# which build is each host serving? they should agree.
curl -s https://karma-connect-9s2z.onrender.com/leads/ | grep -o 'app\.js?v=[a-z0-9]*'
curl -s https://karma-leads-nocodb.onrender.com/leads/ | grep -o 'app\.js?v=[a-z0-9]*'
```

Deploys

`npm start` is `node scripts/leads-migrate.mjs && node server.js`. **Migrations run automatically on every deploy, before the server starts.** Two consequences:

- A bad migration takes the service down, not just the feature. The `&&` is deliberate — a half-migrated database serving traffic is worse than a service that will not start.
- You do not run migrations by hand in production. You push.

```bash
npm run leads:migrate:status   # what has been applied
npm run leads:migrate          # apply (runs on deploy anyway)
```

The build is `vite build` for Connect only. **Karma Leads has no build step** — `leads/public/app.js` is served as written, which is why a Leads fix ships by pushing one file.

Cache-busting is automatic. Do not touch it.

`leads/server/cachebust.js` stamps `?v=` into `index.html` **at serve time**, from a hash of the size and mtime of `app.js`, `app.css` and `na-geo.js`.

The `?v=76`-style numbers you see in the source are markers for a diff and nothing else. This exists because it went wrong: `app.js` shipped twice while `index.html` still said `v=76`, so every browser that had cached that URL kept running the old file — for days, while the fixes appeared to have been deployed.

Never hand-bump a `?v=`. If browsers are stale, the question is whether `cachebust.js` is running, not what number to type.

Environment variables

The authoritative list is what the code reads. Note that `render.yaml` is **out of date** — see the warning at the end of this section.

Cannot start / nothing works without these

VARIABLE	USED BY	WHAT BREAKS WITHOUT IT
`LEADS_DATABASE_URL`	Leads	every Leads route. The module throws at require time
`DATABASE_URL`	Connect	Connect's half
`AUTH_JWT_SECRET`	both	the shared `kc_session` cookie. **Rotating it signs everyone out**
`APP_URL`	digest, mail links	emails ship with dead links

`karmaleads_database` is in **virginia**. Render's *internal* connection string only works from the same region — if the web service is elsewhere, this must be the *external* URL. Getting this wrong looks like a hanging connection, not an auth error.

Also relevant: `PGSSL`, `PGSSLMODE`, `PGCONNECT_TIMEOUT`, `PG_POOL_MAX`.

Access control

VARIABLE	EFFECT
`OWNER_EMAILS`	confers the **owner** tier. Env-only by design — a database edit cannot grant the irreversible powers (ADR-002). Defaults to `tenzinwoesel@karmastaff.com`
`ADMIN_EMAILS`	seeds admins
`ALLOWED_EMAILS`, `ALLOWED_EMAIL_DOMAIN`	lock sign-in down to a list or a domain
`AUTO_INVITE_DOMAIN`	who may self-serve an account

There is no `ADMIN_SIGNUP_CODE`. It was removed: anyone who knew the code could promote themselves.

Mail

VARIABLE	NOTES
`GATEWAY_URL`, `GATEWAY_TOKEN`	the mail gateway. **Not SMTP**
`MAIL_FROM`	the from address

Without these, three features fail *silently in different ways*: team invites send no email, follow-back reminders never arrive, and the digest returns `skipped: "mail is not configured"`. If any one of those is reported, check these first — it is one cause, not three.

Scheduled work

VARIABLE	NOTES
`CRON_TOKEN`	shared secret for `POST /api/cron/daily`. **Unset ⇒ the route returns 503**, refusing rather than defaulting to open
`DIGEST_HOUR`	local hour the digest sends. Default 7
`DIGEST_TZ`	the timezone that hour is judged in
`REPORT_TZ`	the timezone a *day* is measured in for the call rollup and report. Falls back to `DIGEST_TZ`, then `America/New_York`. **Changing it moves the boundary only for days rolled up from then on**, not for rows already stored

The same `CRON_TOKEN` value must also be a **GitHub repository secret**, along with `APP_URL`. See §The morning knock.

Zoho telephony

VARIABLE	NOTES
`ZOHO_VOICE_ENABLED`	master switch
`ZOHO_CLIENT_ID`, `ZOHO_CLIENT_SECRET`	Self Client credentials
`ZOHO_REFRESH_TOKEN`	**a refresh token, not a grant code.** See §Zoho
`ZOHO_ACCOUNTS_DOMAIN`	region, e.g. `accounts.zoho.com`
`ZOHO_DIAL_MODE`	`copy` or the hand-off
`ZOHO_DIAL_URL`	**leave empty.** Empty is the correct production value
`ZOHO_DIAL_COUNTRY_CODE`	

`ZOHO_DIAL_URL` reads `=== undefined` rather than `||`, specifically so that **blanking it in the dashboard takes effect.** With `||`, an empty string was indistinguishable from unset and the old behaviour persisted — which is exactly what happened, and why it looked as if the dashboard was being ignored.

Scrapers and AI

VARIABLE	NOTES
`APIFY_TOKEN`	unset ⇒ job search and LinkedIn research are disabled, not broken
`APIFY_LINKEDIN_COMPANY_ACTOR`, `APIFY_LINKEDIN_PROFILE_ACTOR`	which actor to run
`ANTHROPIC_API_KEY`	read by `api/_proxy.cjs`
`ALLOWED_MODELS`, `MAX_TOKENS_CAP`, `MAX_TOOLS`, `MAX_SEARCH_USES`	the proxy clamps the browser's request; the browser is not trusted
`RATE_LIMIT_RPM`, `DAILY_TOKEN_BUDGET`	spend limits
`BREAKER_FAILS`, `BREAKER_COOLDOWN`	circuit breaker
`RESPONSE_CACHE_TTL`	response cache
`UPSTASH_REDIS_REST_URL`, `UPSTASH_REDIS_REST_TOKEN`	shared rate-limit state. Without them limits are per-instance
`RESEARCH_MODEL`, `RESEARCH_DAILY_CAP`	the briefing
`NWS_USER_AGENT`	the weather service asks for a contact address and may throttle without one

Escape hatches

VARIABLE	EFFECT
`LEADS_ENABLED=0`	unmount Karma Leads with no code change
`LEADS_APP_PATH`	where Leads is served
`KARMA_NO_SWEEP`	disable background sweepers

VARIABLE	EFFECT
`KARMA_DEMO`	demo data
`DEV_AUTH_EMAIL`, `DEV_AUTH_ROLE`	local sign-in bypass. **Never set in production**

⚠️ `render.yaml` no longer describes the service

The blueprint is stale and will mislead you. It still documents **WorkOS as Leads' sign-in** and `SESSION_SECRET` sealing a `kl_session` cookie — that was replaced by the shared `kc_session` JWT. And it does not mention `OWNER_EMAILS`, `CRON_TOKEN`, any `ZOHO_*`, `MAIL_FROM`, `GATEWAY_*`, `NWS_USER_AGENT`, `DIGEST_*`, `RESEARCH_*` or the Apify actor variables — all of which the running service needs.

> Treat the Render dashboard as the truth and this document as the map. `DECISION REQUIRED`: bring `render.yaml` up to date or delete it. A blueprint that would deploy a broken service is worse than no blueprint.

Rotating a secret

1. Add the new value in Render **alongside** the old where the code accepts both; otherwise expect the restart to sign people out.
2. `AUTH_JWT_SECRET` signs out every session in both halves. Do it deliberately, not at 3pm.
3. `CRON_TOKEN` must be changed in **two** places — Render and the GitHub repository secret. Changing one gives a 403 on the next knock.
4. Never paste a placeholder. `YOUR_CLIENT_ID`, `<from the other service>` and a Zoho grant code have all reached production here.

Checks

Is it up?

```bash
curl -s https://karma-connect-9s2z.onrender.com/api/health
```

Free plan: the **first** request after idle takes about 50 seconds to wake the process. A timeout is not an outage. Ask twice.

Is it serving the build you think?

```bash
curl -s https://<host>/leads/ | grep -o 'app\.js?v=[a-z0-9]*'
```

Compare against a local hash of `leads/public/app.js`. If they differ, the deploy has not landed — or you are talking to the old service.

Is the database healthy?

```sql
SELECT count(*) FROM leads WHERE deleted_at IS NULL;         -- ~36,619
SELECT name, applied_at FROM schema_migrations ORDER BY name; -- 10 rows
SELECT pg_size_pretty(pg_database_size(current_database()));  -- 114 MB
```

The full set of diagnostic queries is in The data.

Is CI green?

It exists — `.github/workflows/ci.yml`, on push to `main` and on every PR: `npm ci`, `npm run lint`, `npm test`, `npm run build`. Locally, as of 2026-09-28: lint 0 errors (1 warning), **116 tests passing** in under a second, build clean.

```bash
npm run lint && npm test && npm run build
```

The morning knock

One knock, three jobs. GitHub Actions calls `POST /api/cron/daily` twice a day — `11:00` and `12:00` UTC, because cron is UTC and has no idea about daylight saving, so one of the two is 7am Eastern. That single request does:

JOB	WHAT IT DOES	CLAIMED AS
Storm digest	severe-weather states joined to the lead base, mailed to **everyone**	`storm-digest`
Call report	rolls yesterday into `call_daily`, then mails yesterday's per-person calls and connections to **admins and the owner**	`call-digest`
Follow-back sweep	flushes reminders that fell due overnight	—

Each digest claims its own row in `job_runs`, so a failure in one never silently swallows the other, and the storm digest's "nothing active today, no email" does not stop the call report going out.

The storm digest checks the real local hour in `DIGEST_TZ` and only sends at `DIGEST_HOUR`; the second knock is then a no-op.

The call report differs in one deliberate way: it sends even on a zero day. A silent floor is exactly what a manager needs to be told about, so that email *is* the signal. The storm digest stays quiet on a calm morning, because a daily "nothing happened" is a message people learn to delete.

It reports **yesterday**, not today — at 7am today has barely started. The rollup runs before the send, so the email and the screen read the same stored rows and cannot disagree.

This is external because Render's free plan stops the process after ~15 minutes idle. At 7am nobody has touched the site since the night before, so a `setInterval` inside the app is not running and fires nothing (ADR-003).

Current state: `job_runs` is empty. No scheduled digest has ever been sent.

Diagnosing it

Do this in order — each step rules out the one below.

```bash
# 1. Are the GitHub secrets set? If not the workflow emits a notice and stops.
#    Settings → Secrets and variables → Actions → APP_URL, CRON_TOKEN
gh secret list

# 2. Knock by hand, forced, to one address. force skips the hour check and
#    does NOT claim the day, so it is safe to repeat.
curl -s -X POST "$APP_URL/api/cron/daily" \
  -H 'Content-Type: application/json' -H "x-cron-token: $CRON_TOKEN" \
  -d '{"force":true,"only":"you@karmastaff.com"}'
```

Read the reply. It tells you exactly which gate stopped it:

RESPONSE	MEANING	FIX
`503 CRON_TOKEN is not set`	not set on Render	set it
`403 bad token`	Render and GitHub disagree	make them match
`404`	wrong `APP_URL`	fix the secret
`skipped: "mail is not configured"`	no `GATEWAY_URL` / `GATEWAY_TOKEN`	set them
`skipped: "it is 14:00 in …"`	working correctly, wrong hour	use `force` to test
`skipped: "already sent today"`	working correctly	—
`skipped: "nothing active"`	working correctly — **a quiet day sends no email**	—
`skipped: "weather feed unavailable"`	NWS/FEMA is down or throttling	set `NWS_USER_AGENT`; retry
`{sent: 11, …}`	it worked	—

That last "skipped" is deliberate and worth knowing before you file a bug: **a quiet morning gets no email.** A daily message saying "nothing happened" is a daily message people learn to delete, and then they delete the one that mattered.

```sql
-- 3. did a real (non-forced) run ever land?
SELECT job, ran_on, ran_at, detail FROM job_runs ORDER BY ran_at DESC LIMIT 5;
```

A forced run never writes a row. If you only ever test with `force`, this table stays empty and that is correct.

If the workflow emails a red failure

The workflow is written not to fail when it is merely unconfigured — it emits a notice and stops. "Nobody has set it up yet" is not a broken build, and mailing a red failure twice a day for it only teaches people to ignore the alerts that matter. So a red run means a real failure: read the step output, which prints the HTTP code and the body.

Zoho telephony

Understand this before touching anything: the current blocker is a licence tier, not code. Zoho *Telephony* agents are denied the Voice softphone. No token, scope or configuration change fixes that. The app already degrades correctly — it copies the number to the clipboard and says so.

Every one of the following has been a real incident here.

SYMPTOM	CAUSE	FIX
`Invalid OAuth scope`	the token was minted asking for `ZohoVoice.sdk.READ`	mint with `.ALL`. `npm run zoho:token`
Every token reads as read-only	a literal `0x08` byte inside the scope regex — a shell heredoc ate a backslash, so `\b` became a control character	fixed; found with `od -c`. Do not write regexes through heredocs
`invalid_code`	a **grant code** was pasted into `ZOHO_REFRESH_TOKEN`	exchange the grant code for a refresh token first. The error's `refreshTokenPrefix` identifies this
`Access Denied` on the dialler page	our fallback opened `voice.zoho.com`, the exact page a Telephony licence denies	fixed — copy-only
Windows app-chooser, no Zoho in it	`ZOHO_DIAL_URL=tel:{number}`; Zoho is not a registered `tel:` handler	**blank** `ZOHO_DIAL_URL`
Blanking it changed nothing	`	` treated empty as unset	fixed to `=== undefined`; empty is now the default

Checking the Zoho state

```bash
curl -s "$APP_URL/api/voice/config" -H "Cookie: kc_session=<a real session>"
```

This reports whether Voice is enabled and which scope was actually granted — which is the question, and is not answerable by reading the dashboard.

Outstanding

- `ZOHO_DIAL_URL` should be **deleted** in Render. If it still holds `tel:{number}`, reps get the app-chooser.
- The licence decision is the user's: Zoho Voice seats cost money, and the connect-rate metric in the call report needs one.

Common incidents

"The members can't see the new section"

Almost always the **old service**. Ask for the URL in their address bar before anything else. Then compare the `?v=` each host is serving.

"The follow-back email went to the wrong person"

Fixed. The UI promised "your inbox" while the server used `lead.owner`, so the reminder went to whoever owned the lead. Now uses the caller's identity. Row 5 in `lead_followups` predates the fix.

"The invite didn't send"

`POST /api/users` originally only inserted a row. It now sends mail and the UI reports delivery. If it still fails, the cause is `GATEWAY_URL` / `GATEWAY_TOKEN`.

"A rep has no leads"

Check whether their leads are assigned to an address with no account:

```sql
SELECT l.owner, count(*) FROM leads l
 WHERE l.owner IS NOT NULL AND l.deleted_at IS NULL
   AND NOT EXISTS (SELECT 1 FROM app_users u WHERE lower(u.email)=lower(l.owner))
 GROUP BY 1 ORDER BY 2 DESC;
```

Right now `abhimanyu@karmastaff.com` owns 2,011 leads and has no account. Assignment is free text, not a foreign key.

"A KPI tile says 412 and opens 380 rows"

The predicate is written three times and one copy was changed: `FOCUS` in `leads/server/leads.js`, the `FILTER`s in `leads/server/counts.js`, `FOCUS_TEST` in `leads/public/app.js`. Fix all three.

"Florida numbers look low"

The base stores `FL` (2,585), `Florida` (613) and `Fl` (1) for the same place. Counting one spelling under-reports by about 19%. Every state-aware query must match both.

"A pane only fills half the window"

Happened below 1080px: `.stats-pane` did not span both grid rows. It affected every pane, not one.

The disk is filling

69 MB of the paid 114 MB is abandoned import staging, including two jobs left `open` since 2026-08-11 holding 54,000 rows. See The data before deleting anything.

Working against production safely

There is no staging copy of the 36,619 leads. The development database is production.

- A test row you write is a real row. Give it a distinctive name so you can find it again, and clean it up.
- Prefer read-only probes. Most questions are a `SELECT`.
- `KARMA_DEMO` exists for demonstrations; use it rather than editing real leads.
- Before a migration, read it. The paid plan's backups are the only undo.
- Never point a local server at `LEADS_DATABASE_URL` with `DEV_AUTH_EMAIL` set and then click around as an owner. The destructive routes are real.

After making a change

```bash
npx vitest run          # 116 tests, under a second
npm run lint            # CI runs this; 0 errors expected
npm run build           # only needed if you touched src/
node --check <file>     # on anything a script edited
```

Then say what you changed **and what you verified**. "It should work" is not a report.

PART FOUR · WORKING ON IT

9 Project memory

What to know before editing

Everything here is true of the running system and **not obvious from reading the code**. Most entries exist because something already went wrong.

1. There are two products, two databases, and two user tables

	KARMA CONNECT	KARMA LEADS
Code	`src/` (React + Vite), `server.js`	`leads/` (vanilla JS, **no build step**)
Served at	`/`	`/leads`
Database	`DATABASE_URL` → `karma_connect`	`LEADS_DATABASE_URL` → `karmaleads_database`
Users table	`users` (uuid ids, `is_admin` bool)	`app_users` + `organization_memberships` (int ids, `role` text)

Nothing syncs the two user tables. Making somebody an admin in Connect does not make them an admin in Leads. This is the most common source of "why can't they see X". It is a known wart, not an oversight — the merge was deliberately at the app layer only.

One sign-in covers both: Connect issues an httpOnly `kc_session` JWT cookie (`server/session.cjs`) and Leads reads the same cookie (`leads/server/auth.js`).

2. The role ladder is ranked, and duplicated in three places

`owner > admin > member`. Every gate is "rank at least X", never `=== "admin"`, or an owner loses access to screens they outrank.

Defined in: `leads/server/auth.js` (`RANK`), `server/auth.cjs` (Connect), and `leads/migrations/006_owner_role.sql`. Change one, change all three.

The owner tier lives in `OWNER_EMAILS`, not the database. Deliberate: it must survive a wrong row and must not be grantable from inside the app. Defaults to `tenzinwoesel@karmastaff.com` when unset.

3. Data facts that will bite you

- `leads.owner` **is free text holding email addresses**, matched on equality. Assigning goes through `GET /api/owners` so one person cannot become three owner strings. ~2,000 leads sit under `abhimanyu@karmastaff.com`, who has no account — those leads are invisible in everyone's My tasks.
- **State is stored two ways:** `FL` and `Florida` are both present for the same place. Every query that counts by state must match both spellings or it under-reports Florida by 611. See

`leads/server/alerts.js` → `leadCounts()`.

- **~1 in 5 states is inferred**, not sourced (from city, else area code). 86% coverage at 96% accuracy. Nothing in the UI says so.
- `server/dedupe.js` **is identity.** Franchises are not duplicates. Every clause in it caught a real bad merge. After any change, re-check counts for servpro, stanley steemer, puroclean.
- **Every query over** `leads` **must filter** `deleted_at IS NULL` or binned rows come back as phantom duplicates.
- **Removal ≠ deletion.** `POST /api/leads/:id/remove` is a phone ban that sweeps every lead sharing the number. `DELETE` is the 30-day bin.

---

4. Things that look broken but are correct

- `/login-intro.mp4` **returns 200** — that is the SPA catch-all serving `index.html` for an unknown path. The file was deleted.
- **Everyone shows 0 assigned on the Call report** — accurate. The assigned leads belong to an owner string with no account (see above).
- **Canada is on the Storm watch map and never lights up** — the alert feed is the US National Weather Service. Environment Canada has no API, and the base holds 8 Canadian leads out of 36,619.
- **The Zoho "Notes/Disposition" panel may refuse to save** — that is Zoho's own panel writing to Zoho. Notes typed in Karma Leads are unaffected.

---

5. Zoho is a licence problem, not a code problem

The sales team are **Zoho Telephony** agents. The in-page softphone needs **Zoho Voice**, a different paid product. Without it:

- the softphone cannot authenticate → the app degrades to copying the number
- auto-dial and hang-up-to-advance are unavailable (they need call events)
- the Call report shows calls but not connect rates

`voice.zoho.com` answers those accounts with **Access Denied**, so it must never be used as the fallback — that was a real bug. `ZOHO_DIAL_URL` unset now means *nothing opens*, and the number is copied.

`tel:{number}` is **not** a fix here: checked on a real machine, Zoho was not among the registered `tel:` handlers, so it only produces Windows' app-chooser.

---

6. Do not bump the `?v=` cache-busters by hand

`leads/server/cachebust.js` stamps them at serve time from a hash of the real `app.js` / `app.css` / `na-geo.js` bytes. The numbers left in `index.html` are markers for a diff and are not load-bearing.

This exists because the manual rule failed: `app.js` shipped twice while `index.html` still said `v=76`, so every browser that had cached that URL kept running the old file. Green deploy, correct server, stale

reps.

7. Render is on the free plan, and it sleeps

The process stops after ~15 minutes without a request, so `setInterval` **is not running at 7am**. Anything scheduled must be triggered from outside: `.github/workflows/daily-digest.yml` knocks on `POST /api/cron/daily`, which also flushes follow-back reminders that came due overnight.

The two cron times (11:00 and 12:00 UTC) exist so exactly one lands at 7am local through daylight saving; `job_runs` (migration 009) makes the day something you claim once.

8. Schema changes only via `migrations/` + `node leads/server/migrate.js`

Ten migrations so far. `karma_api` is the only DDL role. Never hand-edit the schema on a running database.

9. There is a LinkedIn automation bot in `agent/`

It drives a real browser against LinkedIn from a **local machine** (never the server — `render.yaml` omits Playwright deliberately). Server-side caps in `server/linkedin-agent.cjs`: 25/day, 8/hour, 45–180s gaps, weekdays 09:00–17:00, counted from the database so restarting resets nothing.

`agent/.profile` holds a live signed-in LinkedIn session. It is gitignored. The repo owner built it and accepts the ToS risk knowingly — this is context, not a decision to revisit.

10. Known limitations, stated plainly

LIMITATION	WHY IT IS STILL LIKE THIS
Two user tables, two role stores	The merge was app-layer only, by request. Unifying is a contained job nobody has authorised.
Call outcomes are client-reported	Only the browser holding the softphone knows a call ended. A rep dialling from a handset produces no rows.
Research is one lookup, not monitoring	Briefings are bought once and stored; nothing re-checks them.
Connect's own data is unavailable locally	No `DATABASE_URL` in local `.env`; the dev bypass synthesises a user so the UI renders.
The old service is still live	`karma-leads-nocodb.onrender.com` serves frozen old code against the *same* database. Reps who kept the bookmark see a two-week-old app.

11. Do-not-change without understanding

- `server/dedupe.js` — identity rules, each clause earned
- `leads/server/auth.js` `resolve()` — the shared-cookie path both products depend on
- `api/_proxy.cjs` — the Anthropic proxy, rate limits, circuit breaker, cost recording
- The partial unique indexes in migrations 008 and 010 — they are what stop duplicate reminders and duplicate call sessions

10 Rules for changing this codebase

Aimed at an agent; they are just good rules

Aimed at an AI coding agent, but they are just good rules.

Before you touch anything

1. **Read Project memory.** It is short and it is mostly things that already went wrong.
2. **Read the nearest `CLAUDE.md`.** `leads/CLAUDE.md` holds the invariants for the half of the system with the data in it.
3. **Work out which half you are in.** Connect and Leads have different languages, different databases, different user tables and different design vocabularies. A fix in one is not a fix in the other.

While you work

Check, do not assume. Nearly every wrong turn in this project's history was a confident guess:

- "the deploy is stuck" — it had deployed; the cache-buster hadn't
- "the token is fine" — Render held the grant code, not the refresh token
- "members can't see the new sections" — they were on the old service
- "`tel:` will reach Zoho" — Zoho was not a registered handler

Each was one command away from being known. Run the command.

Verify against the running system, not the source. `curl` the endpoint, query the database, drive the browser. The source tells you what should happen.

A failing test or a red screenshot is the finding. Do not explain it away. Twice in this project a test "failure" was the test being wrong — both times that was established by capturing the actual DOM and the actual response, not by reasoning about it.

Do not write through a shell heredoc what you can write with a file tool. Two real bugs came from heredocs eating backslashes: a literal `0x08` inside a regex that made every OAuth token look read-only, and a newline inside a string literal that broke the build. If you must, verify with `od -c`.

Rules with teeth

RULE	BECAUSE
Schema changes only via `leads/migrations/` + `migrate.js`	`karma_api` is the only DDL role; hand-edits drift
Every `leads` query filters `deleted_at IS NULL`	or binned rows return as phantom duplicates
Role checks are rank comparisons, never `=== "admin"`	or an owner loses screens they outrank

RULE	BECAUSE
Counting by state matches both `FL` and `Florida`	or Florida is under-reported by 611
Never hand-bump the `?v=` cache-busters	`cachebust.js` does it from the file bytes
Never commit a secret; `.env` is gitignored	tokens in this project are durable keys to a phone system
Never put a placeholder in a config value	`YOUR_CLIENT_ID`, `<from the other service>` and a grant code have all been pasted into production here
Do not add a dependency without saying why	the runtime is twelve packages and that is a feature
Clean up test rows you write to production	this repo's dev database *is* production

Things that are load-bearing

Change these only with a reason that survives reading the comments in them:

- `server/dedupe.js` — identity. Franchises are not duplicates. Every clause caught a real bad merge.
- `leads/server/auth.js` `resolve()` — the shared-cookie path both products depend on.
- `api/_proxy.cjs` — Anthropic proxy: rate limits, circuit breaker, cost recording.
- The partial unique indexes in migrations 008 and 010 — they are what stop duplicate reminders and duplicate call sessions.

When you finish

- `npx vitest run` — 116 tests, under a second
- `npx vite build` if you touched `src/`
- `node --check` on any file a script edited
- Say what you changed and what you verified. If you could not verify something, say that too — "it should work" is not a report.

On scope

The instinct to tidy is usually wrong here. Karma Leads has no build step and one 5,700-line `app.js`; that is why it survived being moved between two hosting setups and why a fix ships by pushing a file. Modernising it is a project, not a side-effect of a bug fix.

Equally: do not narrow the job. If something is broken in three places, fix three places and say so.

11 ADR-001

Merge at the app layer, not the data layer

Status Accepted · **Date** 2026-09 · **Owner** repo owner

Context

Karma Connect and Karma Leads were separate products on separate Render services with separate databases. Two services meant two bills for one team.

Options

1. **Full merge** — one codebase, one database, one user table.
2. **App-layer merge** — one web service mounting both, databases untouched.
3. **Leave them** — keep paying for two.

Decision

Option 2. `leads/server/index.js` exports `mount(app, opts)`; `server.js` calls it. Connect serves `/`, Leads serves `/leads`, one sign-in cookie covers both. API paths are identical merged or standalone, so the Leads front end needed no changes.

Reason

The owner was explicit: *"i dont want to merge them fulll ... i want them both in one website"*. The saving is the second **web service**; the Leads database is the paid one and has to stay regardless, so a data merge would have bought nothing it did not already have.

Tradeoffs

- Two user tables that nothing syncs — admin must be granted twice. This is the system's main known wart (PROJECT-MEMORY §1).
- Two design vocabularies that look like siblings rather than one product.
- Against that: no data migration, no risk to 36,619 lead rows, and it shipped.

What would change it

The team growing past ~15 people, where granting access twice starts costing more than the migration would.

12 ADR-002

The owner tier lives in the environment

Status Accepted · **Date** 2026-09 · **Owner** repo owner

Context

`admin` had grown to mean two unrelated things: running the sales floor (assigning, importing, reading the team's numbers) and holding the keys (appointing other admins, deleting accounts, purging the bin). One role could not say both, and any admin could mint admins — so one compromised manager account could lock the real owner out.

Options

1. A third role stored in `organization_memberships.role` like the others.
2. A third role in an environment variable (`OWNER_EMAILS`).
3. Both: stored, but the environment wins.

Decision

Option 3. `OWNER_EMAILS` is authoritative; a stored `'owner'` row is honoured so the team list agrees with the session, but the environment overrides it. Defaults to `tenzinwoesel@karmastaff.com` when unset.

Reason

The top tier must survive a database that says otherwise — a bad UPDATE, a restored backup, someone with SQL access — and must not be grantable from inside the app, or it is just another admin. An environment list is the smallest thing with both properties.

Defaulted rather than left empty: an unset variable quietly meaning "nobody owns this" is how the previous identity provider became unadministrable.

Tradeoffs

- Handing over ownership needs a deploy, not a click. Intended.
- Demoting a listed owner in the UI changes the row and nothing else — so the API returns a warning saying exactly that.

Related

Replaced `ADMIN_SIGNUP_CODE`, a single shared string that turned whoever typed it into an admin, could not be revoked for one person, and logged nothing.

13 ADR-003

Scheduled work is triggered from outside

Status Accepted · **Date** 2026-09 · **Owner** repo owner

Context

The 7am storm digest has to send at 7am. The Render service is on the free plan, which stops the process after ~15 minutes without a request. At seven in the morning nobody has touched the site since the night before, so `setInterval` is not running and fires nothing.

This also silently affected follow-back reminders, which used the same pattern.

Options

1. Upgrade to a paid always-on instance.
2. Render Cron Job (a separate paid service type).
3. An external scheduler knocking on an endpoint.

Decision

Option 3. `.github/workflows/daily-digest.yml` posts to `/api/cron/daily` with a shared secret. The knock also flushes any follow-back reminders that came due overnight — the same problem, solved by the same wake-up.

Reason

Free, and the repo is already on GitHub. The whole project exists to cut hosting cost; paying for always-on to send one email would be self-defeating.

Tradeoffs

- The endpoint needs its own auth (`CRON_TOKEN`), refused outright when unset rather than defaulting to open.
- Cron is UTC and knows nothing of daylight saving, so **two** times are fired (11:00 and 12:00 UTC) and the server decides which is 7am local. `job_runs` (migration 009) makes the day something you claim once, in a single statement that cannot race.
- If GitHub Actions is down, the digest is missed. Acceptable for a sales briefing; would not be for anything transactional.

14 ADR-004

Map geometry is generated offline

Status Accepted · **Date** 2026-09 · **Owner** repo owner

Context

Storm watch needs a map of the US and Canada. Karma Leads has no build step, a strict Content-Security-Policy, and is served from one Render instance.

Options

1. A map library from a CDN (Leaflet/d3) plus a tile server.
2. Inline SVG from geometry projected at runtime.
3. Geometry projected and simplified **offline**, shipped as a plain script.

Decision

Option 3. `leads/scripts/build-na-geo.mjs` turns Natural Earth 1:50m admin-1 (public domain) into `leads/public/na-geo.js`, which sets `window.NA_GEO`. 2.3 MB of source becomes 52 KB via Albers equal-area projection, Douglas-Peucker simplification and dropping sub-threshold rings.

Reason

No map library, no tile server, nothing new for the CSP to allow, and no runtime cost. The file is generated — re-run the script rather than editing it.

Tradeoffs

- Regenerating needs the source GeoJSON, which is not vendored. The script prints its own usage.
- Equal-area, not Mercator: on a Mercator, Nunavut would out-size the entire lower 48 and a reader would take that as meaning something.
- The frame is cropped to the US with headroom, because fitting all of Canada puts 3,000 km of empty Arctic on screen and squeezes the states where every lead is into a strip.
- DC is kept below the speck threshold on purpose: it is a jurisdiction the rest of the app filters by, and a map missing a place lies by omission.

PART SIX · WHAT IS STILL OWED

15 Outstanding

What is missing, what needs a decision, what was fixed in passing

Written documentation that is wrong is worse than none, because people trust it. This file says what is missing and what each section needs before it is worth writing.

Written

DOCUMENT	STATUS
`README.md`	done
`PROJECT.md`	done
`architecture/ARCHITECTURE.md`	done
`architecture/DATABASE.md`	done — all 21 Leads tables, verified against the live database
`operations/RUNBOOK.md`	done — every env var, every incident seen so far
`architecture/API.md`	done — **generated** by `scripts/gen-api-docs.mjs`, 133 routes, checked in CI
`architecture/SECURITY.md`	done — including a gaps table
`architecture/INTEGRATIONS.md`	done — ten services, each with its failure mode
`product/FEATURES.md`	done — 17 Leads features, 12 Connect views, with acceptance criteria
`ai/PROJECT-MEMORY.md`	done
`ai/AI-RULES.md`	done
`decisions/ADR-001` … `ADR-004`	done

Next

Every document planned in the first pass is now written. What remains is not documentation — it is the actions and decisions listed below, which are the things documenting the system surfaced.

If more documentation is wanted, the candidates in rough value order are:

DOCUMENT	NEEDS	NOTES
`architecture/FRONTEND.md`	nothing	The 5,700-line `app.js`: the `S` state object, `fromApi()`, keyset paging, where to add a screen. Useful to anyone who has to touch Leads
`operations/ONBOARDING.md`	nothing	What a new starter needs on day one, including that they must be added to **two** user tables
`decisions/ADR-005…`	a decision to record	ADRs are written when a decision is made, not in advance

Deliberately still not written: the list at the end of this file.

Corrections made to earlier drafts

Recorded rather than quietly fixed, because the pattern matters more than the facts.

CLAIM	REALITY
"There is no CI"	`.github/workflows/ci.yml` has existed since the *Upgrade Phase 1* commit. Lint, test, build, on push to `main` and every PR. Green
"104 tests"	116
`leads/CLAUDE.md`: "Auth: WorkOS AuthKit, `kl_session`"	Shared `kc_session` HS256 JWT from Connect; WorkOS is a fallback only. Corrected in place — AI-RULES sends agents to that file, so it was actively misleading them
`job_runs` empty ⇒ "the knock is not arriving"	Too strong. The day is claimed *after* the hour and mail checks and a `force`d run never claims, so five different states produce an empty table. RUNBOOK distinguishes them

Each was asserted from memory instead of checked. All four were one command away.

Findings that need a decision or an action

`DECISION REQUIRED` — 2,011 leads assigned to nobody

`abhimanyu@karmastaff.com` owns 2,011 leads — 6% of the base, the largest block of *assigned* leads — and has no row in `app_users`. Nobody can see them; the one follow-up booked against that owner can never be delivered. Either create the account or reassign. This is dark inventory, not a cosmetic problem.

`ACTION` — 69 MB of abandoned import staging on a paid plan

60% of the 114 MB database. Two import jobs left `open` since 2026-08-11 holding 54,000 staging rows, plus committed jobs whose rows were never purged. Checklist in The data. There is no automatic purge and there probably should be.

`ACTION` — no scheduled digest has ever been sent

`job_runs` is empty. Most likely the GitHub repository secrets (`APP_URL`, `CRON_TOKEN`) are unset, in which case the workflow emits a notice and stops by design. Diagnosis is five minutes: RUNBOOK §The morning knock.

`DECISION REQUIRED` — `render.yaml` would deploy a broken service

It still documents WorkOS as Leads' sign-in and `SESSION_SECRET` sealing `kl_session`, and omits `OWNER_EMAILS`, `CRON_TOKEN`, every `ZOHO_*`, `MAIL_FROM`, `GATEWAY_*`, `NWS_USER_AGENT`, `DIGEST_*`, `RESEARCH_*` and the Apify actors. Recommended: update it, or delete it. A blueprint that lies is worse than none.

`DECISION REQUIRED` — Connect has no migrations, and has already drifted

Tables are created at boot across three files, and `db/schema.sql` also exists. Nine tables are defined twice. `message_sends` genuinely differs between the two, patched over by three `add column if not exists` statements. `db/schema.sql` is not even complete — `activity`, `linkedin_agent` and `outreach_queue` exist only in code, so a database built from that file would have no send queue.

Recommended: **delete** `db/schema.sql` **now** (an afternoon, removes a file that is already lying), adopt the Leads migration runner later.

`ACTION` — three dead NocoDB tables

`notification`, `workspace`, `workspace_user`. One row each, read by nothing. `workspace.stripe_customer_id` keeps prompting the question of whether this product takes payments. It does not. Dropping them is a migration and it is safe.

`DECISION REQUIRED` — retire the old service

`karma-leads-nocodb.onrender.com` is still live, serving frozen code against the same database. It is why reps reported the new sections were missing. Suspend it after the checklist in RUNBOOK.

`DECISION REQUIRED` — unify the two user tables

Named in `PROJECT-MEMORY.md` as the known wart. Every new starter is added twice and the two can silently disagree. Recommended: do it, before the team grows past ~15.

`DECISION REQUIRED` — one design system or two

Connect and Leads have separate colour, spacing and component vocabularies. They already look like siblings rather than one product. Options: (a) leave them, document both, accept the seam; (b) extract shared tokens and converge slowly; (c) unify now. Recommended **(a)** — the seam costs nothing operationally and (c) is weeks of work with no user-facing gain.

Fixed in passing

`api/_proxy.cjs` **contained two raw NUL bytes.** They were deliberate — a delimiter in a cache-key hash — but written as literal bytes rather than `\u0000` escapes, which made `grep` classify the file as binary and **skip it.** Consequence: a repo-wide grep silently missed 17 environment variables that only this file reads, including `ANTHROPIC_API_KEY` and `UPSTASH_REDIS_REST_TOKEN`. A secret sweep over this repo would have missed them too.

Replaced with `\u0000`. Identical string to JavaScript, so cache keys are unchanged; 116 tests pass; the file is now text and greppable.

Deliberately not written, and why

SECTION THE BRIEF ASKED FOR	WHY NOT
Payments	The product takes no payments. The Stripe columns are NocoDB residue
SMS / push notifications	Neither exists. Email only
Mobile app / device matrix	There is no mobile app, and the app is used on desktop by a sales floor. A responsive audit is worth doing; a device support matrix is not
Personas (fictional)	The users are eleven named people at one company. Inventing personas would be theatre. `PROJECT.md` describes the three real roles

SECTION THE BRIEF ASKED FOR	WHY NOT
SEO	Both products are behind a sign-in. There is nothing to index
i18n	One company, one country, one language
Design tokens as one system	Leads has a real token set in `app.css`; Connect has its own in `styles.css`. They are **not** the same system. Documenting them as one would be a lie